



河南大學

Henan University

Dissonant Interval

Contestant

曹家宁

Jianing Cao

罗施达

Shida Luo

孙小文

Xiaowen Sun

目录

1 数学	1		
1.1 数论	1		
1.1.1 线性筛	1		
1.1.2 MillerRabin	1		
1.1.3 PollardRho	2		
1.1.4 区间筛	3		
1.1.5 中国剩余定理	4		
1.1.6 扩展中国剩余定理	4		
1.2 线性代数	4		
1.2.1 线性基	4		
1.2.2 矩阵	5		
1.3 组合数学	6		
1.3.1 组合数	6		
1.3.2 卢卡斯定理	6		
1.4 多项式	7		
1.4.1 FFT	7		
1.4.2 NTT	8		
1.4.3 Poly	9		
1.5 取模类	11		
2 图论	11		
2.1 图的连通性	11		
2.1.1 Tarjan 割点	11		
2.1.2 Tarjan 割边	12		
2.1.3 Tarjan 强连通分量	12		
2.1.4 Tarjan 点双连通分量	13		
2.1.5 Tarjan 边双连通分量	14		
2.2 重链剖分	14		
2.3 长链剖分	15		
2.4 树的重心	17		
2.5 欧拉回路	17		
2.6 流和匹配	19		
2.6.1 EdmondsKarp	19		
2.6.2 二分图判定	19		
2.6.3 二分图最大匹配	20		
3 数据结构	21		
3.1 树状数组	21		
3.1.1 树状数组	21		
3.1.2 树状数组 2	21		
3.2 线段树	22		
3.2.1 线段树 simple	22		
3.2.2 线段树	23		
3.2.3 线段树优化建图	25		
3.2.4 动态开点线段树	26		
3.2.5 主席树	29		
3.2.6 标记永久化主席树	30		
3.3 ST 表	32		
3.4 并查集	32		
3.5 可撤销并查集	33		
3.6 带权并查集	34		
3.7 左偏树	34		
3.8 智慧集	35		
3.9 笛卡尔树	36		
3.10 波纹疾走树	37		
3.11 欧拉序	38		
4 字符串	39		
4.1 字符串哈希	39		
4.2 字典树	40		

4.3	KMP	41
4.4	EXKMP	41
4.5	马拉车	42
5	计算几何	42
5.1	凸包	42
6	杂项	46
6.1	取整	46
6.2	日期公式	46
6.3	动态 bitset	47
6.4	高精度	49
6.5	命令行	51
7	附录	51
7.1	编译参数	51
7.2	OJ 测试	51
7.3	随机素数	52
7.4	组合数学公式	52
7.5	常数表	52

1 数学

1.1 数论

1.1.1 线性筛

```
1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 constexpr int MAXN = 1E8;
5 struct Sieve {
6     Sieve(int n) : n(n), mnp(n + 1) {
7         for(int i = 2; i <= n; ++i) {
8             if(mnp[i] == 0) {
9                 mnp[i] = i;
10                prime.push_back(i);
11            }
12            for(int j = 0; i * prime[j] <= n; ++j) {
13                mnp[i * prime[j]] = prime[j];
14                if(mnp[i] == prime[j]) break;
15            }
16        }
17    }
18    bool isPrime(int x) {
19        return mnp[x] == x;
20    }
21    int minp(int x) {
22        return mnp[x];
23    }
24    int phi(int x) { //欧拉函数
25        static auto _phi = build([&](auto &f, int i, int p, int j) {
26            f[i] = f[j] * (p == mnp[j] ? p : p - 1);
27        });
28        return _phi[x];
29    }
30    int mu(int x) { //莫比乌斯函数
31        static auto _mu = build([&](auto &f, int i, int p, int j) {
32            f[i] = (p == mnp[j] ? 0 : -f[j]);
33        });
34        return _mu[x];
35    }
36    int d(int x) { //约数个数函数
37        static auto _d = build([&](auto &f, int i, int p, int j) {
38            static std::vector<int> a(n + 1);
39            a[i] = (p == mnp[j] ? a[j] + 1 : 1);
40            f[i] = (p == mnp[j] ? f[j] / (a[j] + 1) * (a[i] + 1) : f[j] * 2);
41        });
42        return _d[x];
```

```
43    }
44    int sigma(int x) { //约数和函数
45        static auto _sigma = build([&](auto &f, int i, int p, int j) {
46            static std::vector<i64> s(n + 1);
47            s[i] = (p == mnp[j] ? s[j] * p : p);
48            f[i] = f[j] + s[i] * (p == mnp[j] ? f[j] / s[j] : f[j]);
49        });
50        return _sigma[x];
51    }
52    template<typename T = int, typename Func>
53    std::vector<T> build(Func& f) {
54        std::vector<T> f(n + 1);
55        f[1] = 1;
56        for(int i = 2; i <= n; ++i) {
57            f[i] = f[i] * (p == mnp[i] ? f[i] / s[i] : f[i]);
58        }
59        return f;
60    }
61    int n;
62    std::vector<int> mnp, prime;
63 } sieve(MAXN);
64
65 int main() {
66     std::ios::sync_with_stdio(false);
67     std::cin.tie(nullptr);
68
69     return 0;
70 }
```

1.1.2 MillerRabin

模板题: [Luogu SP288](#)

时间复杂度: $O(k \log^3(n)), k = 7$

```
1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 i64 qpow(i64 a, i64 b, i64 p) {
5     i64 res = 1;
6     while(b) {
7         if(b & 1) {
8             res = (__int128)res * a % p;
9         }
10        a = (__int128)a * a % p;
11        b >>= 1;
12    }
13    return res;
14 }
```

```

15
16 bool Minller(i64 n) {
17     if(n <= 1 || n % 2 == 0) return (n == 2);
18     i64 u = n - 1, k = 0;
19     while(u % 2 == 0) u /= 2, ++k;
20     static std::vector<i64> base = {2, 325, 9375, 28178, 450775, 9780504,
1795265022};
21     for(auto x : base) {
22         i64 res = qpow(x, u, n);
23         if(res == 0 || res == 1 || res == n - 1) continue;
24         for(int i = 1; i <= k; ++i) {
25             res = (__int128)res * res % n;
26             if(res == n - 1) break;
27             if(i == k) return false;
28         }
29     }
30     return true;
31 }
32
33 void solve() {
34     i64 x;
35     std::cin >> x;
36     std::cout << (Minller(x) ? "YES" : "NO") << '\n';
37 }
38
39 int main() {
40     std::ios::sync_with_stdio(false);
41     std::cin.tie(nullptr);
42     int T = 1;
43     std::cin >> T;
44     while(T--) {
45         solve();
46     }
47     return 0;
48 }

```

1.1.3 PollardRho

模板题: Factorize

时间复杂度: $O(n^{\frac{1}{4}})$

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 i64 qpow(i64 a, i64 b, i64 p) {
5     i64 res = 1;
6     while(b) {
7         if(b & 1) {

```

```

8             res = (__int128)res * a % p;
9         }
10        a = (__int128)a * a % p;
11        b >>= 1;
12    }
13    return res;
14 }
15
16 //Miller_rabin判断质数
17 bool Miller(i64 n) {
18     if(n <= 1 || n % 2 == 0) return (n == 2);
19     i64 u = n - 1, k = 0;
20     while(u % 2 == 0) u /= 2, ++k;
21     static std::vector<i64> base = {2, 325, 9375, 28178, 450775, 9780504,
1795265022};
22     for(auto x : base) {
23         i64 res = qpow(x, u, n);
24         if(res == 0 || res == 1 || res == n - 1) continue;
25         for(int i = 1; i <= k; ++i) {
26             res = (__int128)res * res % n;
27             if(res == n - 1) break;
28             if(i == k) return false;
29         }
30     }
31     return true;
32 }
33
34 //Pollard_rho找因子
35 i64 Pollard_rho(i64 n) {
36     assert(n >= 2);
37     if (!(n & 1)) return 2;
38     auto f = [&](i64 x) {
39         return (__int128)x * x % n + 5;
40     };
41     i64 x = 0, y = 0, prod = 1;
42     for (int i = 30, z = 0; i % 64 || std::gcd(prod, n) == 1; ++i) {
43         if (x == y) {
44             x = ++z;
45             y = f(x);
46         }
47         i64 q = (__int128)prod * (x + n - y) % n;
48         if (q) prod = q;
49         x = f(x), y = f(f(y));
50     }
51     return std::gcd(prod, n);
52 }
53
54 std::vector<i64> factorize(i64 x) {

```

```

55     std::vector<i64> res;
56     auto f = [&](auto f, i64 x) ->void {
57         if (x == 1) return;
58         if (Miller(x)) {
59             res.push_back(x);
60             return;
61         }
62         i64 y = Pollard_rho(x);
63         f(f, y), f(f, x / y);
64     };
65     f(f, x);
66     std::ranges::sort(res);
67     return res;
68 }
69
70 void solve() {
71     i64 x;
72     std::cin >> x;
73     auto res = factorize(x);
74     std::cout << res.size() << " \n"[res.empty()];
75     for(int i = 0; i < res.size(); ++i) {
76         std::cout << res[i] << " \n"[i + 1 == res.size()];
77     }
78 }
79
80 int main() {
81     std::ios::sync_with_stdio(false);
82     std::cin.tie(nullptr);
83     int T = 1;
84     std::cin >> T;
85     while(T--) {
86         solve();
87     }
88     return 0;
89 }

```

1.1.4 区间筛

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 constexpr int MAXN = 2E5;
5 std::vector<int> prime;
6 std::vector<bool> nonPrime(MAXN + 1);
7 void findPrime(int n) {
8     nonPrime[0] = nonPrime[1] = 1;
9     for(int i = 2; i <= n; ++i) {
10         if(nonPrime[i] == false) {

```

```

11             prime.push_back(i);
12         }
13         for(int j = 0; i * prime[j] <= n; ++j) {
14             nonPrime[i * prime[j]] = true;
15             if(i % prime[j] == 0) break;
16         }
17     }
18 }
19
20 //区间筛, 筛区间[L, R]的质数
21 //https://www.luogu.com.cn/problem/UVA10140
22 int main() {
23     i64 L, R;
24     findPrime(MAXN);
25     while(std::cin >> L >> R) {
26
27         std::vector<i64> res;
28         std::vector<bool> nonp(R - L + 1);
29         for(auto x : prime) {
30             if(x > R) break;
31             for(int j = std::max((L + x - 1) / x, 2LL); 1LL * j * x <= R; ++j) {
32                 nonp[j * x - L] = 1;
33             }
34         }
35         for(int i = 0; i <= R - L; ++i) {
36             if(nonp[i] == 0 && i + L >= 2) {
37                 res.push_back(i + L);
38             }
39         }
40
41         i64 mn = INT_MAX, mx = INT_MIN;
42         int mnidx = -1, mxidx = -1;
43         for(int i = 1; i < res.size(); ++i) {
44             if(res[i] - res[i - 1] < mn) {
45                 mn = res[i] - res[i - 1];
46                 mnidx = i;
47             }
48             if(res[i] - res[i - 1] > mx) {
49                 mx = res[i] - res[i - 1];
50                 mxidx = i;
51             }
52         }
53         if(res.size() <= 1) {
54             std::cout << "There are no adjacent primes.\n";
55         } else {
56             std::cout << res[mnidx - 1] << ',' << res[mnidx] << " are closest, "
57                 << res[mxidx - 1] << ',' << res[mxidx] << " are most distant.\n";
58         }
59     }

```

```

58     }
59 }
60     return 0;
61 }

```

1.1.5 中国剩余定理

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 std::array<i64, 3> exgcd(i64 a, i64 b) {
5     if(b == 0) {
6         return {a, 1, 0};
7     }
8     auto [gd, x, y] = exgcd(b, a % b);
9     return {gd, y, x - a / b * y};
10 }
11
12 i64 CRT(std::vector<std::pair<i64, i64>> &v) {
13     i64 M = 1;
14     for(const auto &[m, r] : v) {
15         M *= m;
16     }
17     i64 ans = 0;
18     for(const auto &[m, r] : v) {
19         i64 Mi = M / m;
20         auto [gd, x, y] = exgcd(Mi, m);
21         ans += (__int128)r * Mi % M * x % M;
22     }
23     return (ans % M + M) % M;
24 }
25
26 int main() {
27     std::ios::sync_with_stdio(false);
28     std::cin.tie(nullptr);
29     int n;
30     std::cin >> n;
31     std::vector<std::pair<i64, i64>> v(n);
32     for(auto &[m, r] : v) {
33         std::cin >> m >> r;
34     }
35     std::cout << CRT(v) << '\n';
36     return 0;
37 }

```

1.1.6 扩展中国剩余定理

```

1 #include <bits/stdc++.h>

```

```

2 using i64 = long long;
3
4 std::array<i64, 3> exgcd(i64 a, i64 b) {
5     if(b == 0) {
6         return {a, 1, 0};
7     }
8     auto [gd, x, y] = exgcd(b, a % b);
9     return {gd, y, x - a / b * y};
10 }
11
12 i64 EXCRT(std::vector<std::pair<i64, i64>> &v) {
13     i64 M = v[0].first, ans = v[0].second;
14     for(int i = 1; i < v.size(); ++i) {
15         auto [m, r] = v[i];
16         auto [gd, x, y] = exgcd(M, m);
17         i64 dx = ((ans - r) % m + m) % m;
18         if(dx % gd != 0) return -1;
19         i64 k = m / gd;
20         x = __int128(dx / gd) % k * x % k;
21         ans -= x * M;
22         M *= k;
23         ans = (ans % M + M) % M;
24     }
25     return ans;
26 }
27
28 int main() {
29     std::ios::sync_with_stdio(false);
30     std::cin.tie(nullptr);
31     int n;
32     std::cin >> n;
33     std::vector<std::pair<i64, i64>> v(n);
34     for(auto &[m, r] : v) {
35         std::cin >> m >> r;
36     }
37     std::cout << EXCRT(v) << '\n';
38     return 0;
39 }

```

1.2 线性代数

1.2.1 线性基

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template<typename T, int N = std::numeric_limits<T>::digits>
5 struct Basis {

```

```

6  bool insert(T x) {
7      for(int i = N - 1; i >= 0; --i) {
8          if(x >> i & 1) {
9              if(!base[i]) {
10                 base[i] = x;
11                 _count++;
12                 return true;
13             }
14             x ^= base[i];
15         }
16     }
17     flag = true;
18     return false;
19 }
20 T kth(T k) {
21     k -= flag;
22     T res{};
23     int len = _count - 1;
24     for (int i = N - 1; i >= 0; i--) {
25         if(!base[i]) continue;
26         if ((k >> len & 1) ^ (res >> i & 1)) {
27             res ^= base[i];
28         }
29         len--;
30     }
31     return res;
32 }
33 T querymax() {
34     T res{};
35     for(int i = N - 1; i >= 0; --i) {
36         res = std::max(res ^ base[i], res);
37     }
38     return res;
39 }
40 int count() {
41     return _count;
42 }
43 T size() {
44     return ((T)1 << count()) - (T)!flag;
45 }
46 bool flag = false;
47 std::array<T, N> base{};
48 int _count{};
49 };
50
51 void solve() {
52     int n;
53     std::cin >> n;

```

```

54     Basis<i64> basis;
55     for(int i = 0; i < n; ++i) {
56         i64 x;
57         std::cin >> x;
58         basis.insert(x);
59     }
60     std::cout << basis.querymax() << '\n';
61 }
62
63 int main() {
64     std::ios::sync_with_stdio(false);
65     std::cin.tie(nullptr);
66     int T = 1;
67     // std::cin >> T;
68     while(T--) {
69         solve();
70     }
71     return 0;
72 }

```

1.2.2 矩阵

```

1  #include <bits/stdc++.h>
2  using i64 = long long;
3
4  template<typename T>
5  struct Matrix {
6      Matrix() : Matrix(0, 0) {};
7      Matrix(int _n, int _m, T val = T{}) : n(_n), m(_m), mt(n * m, val){}
8      Matrix(const std::initializer_list<std::initializer_list<T>> &v) : Matrix(v.size
9          (), v.begin()->size()) {
10         int i = 0;
11         for(auto &row : v) {
12             assert(row.size() == m);
13             for(auto &x : row) {
14                 mt[i++] = x;
15             }
16         }
17     }
18     std::span<T> operator[](int idx) {
19         return std::span<T>(mt.data() + idx * m, m);
20     }
21     std::span<const T> operator[](int idx) const {
22         return std::span<const T>(mt.data() + idx * m, m);
23     }
24     friend Matrix<T> operator*(const Matrix<T>& A, const Matrix<T>& B) {
25         assert(A.m == B.n);
26         Matrix<T> MT(A.n, B.m);

```

```

26     for(int i = 0; i < MT.n; ++i) {
27         for(int j = 0; j < MT.m; ++j) {
28             for(int k = 0; k < A.m; ++k) {
29                 MT[i][j] = MT[i][j] + A[i][k] * B[k][j];
30             }
31         }
32     }
33     return MT;
34 }
35 friend Matrix<T> operator+(const Matrix<T>& A, const Matrix<T>& B) {
36     assert(A.n == B.n && A.m == B.m);
37     Matrix<T> MT(A.n, B.m);
38     for(int i = 0; i < MT.n; ++i) {
39         for(int j = 0; j < MT.m; ++j) {
40             MT[i][j] = A[i][j] + B[i][j];
41         }
42     }
43     return MT;
44 }
45 friend Matrix<T> operator-(const Matrix<T>& A, const Matrix<T>& B) {
46     assert(A.n == B.n && A.m == B.m);
47     Matrix<T> MT(A.n, B.m);
48     for(int i = 0; i < MT.n; ++i) {
49         for(int j = 0; j < MT.m; ++j) {
50             MT[i][j] = A[i][j] - B[i][j];
51         }
52     }
53     return MT;
54 }
55 static Matrix<T> eye(int n) {
56     Matrix<T> MT(n, n);
57     for(int i = 0; i < n; ++i) {
58         MT[i][i] = 1;
59     }
60     return MT;
61 }
62 static Matrix<T> qpow(Matrix<T> a, i64 b) {
63     Matrix<T> res(Matrix<T>::eye(a.n));
64     while(b != 0) {
65         if(b & 1) {
66             res = res * a;
67         }
68         a = a * a;
69         b >>= 1;
70     }
71     return res;
72 }
73 friend std::ostream& operator<<(std::ostream& os, const Matrix<T>& o) {

```

```

74     for(int i = 0; i < o.n; ++i) {
75         for(int j = 0; j < o.m; ++j) {
76             os << o[i][j] << " \n"[j + 1 == o.m];
77         }
78     }
79     return os;
80 }
81 int n, m;
82 std::vector<T> mt;
83 };
84
85 int main() {
86     std::ios::sync_with_stdio(false);
87     std::cin.tie(nullptr);
88
89     return 0;
90 }

```

1.3 组合数学

1.3.1 组合数

```

1 constexpr i64 P = 998244353;
2 constexpr i64 MAXN = 3000;
3
4 struct Comb {
5     Comb(int n) : _fac(n + 1, 1), _inv(n + 1, 1), _invfac(n + 1, 1) {
6         for(int i = 2; i <= n; ++i) {
7             _fac[i] = _fac[i - 1] * i % P;
8             _inv[i] = (P - P / i) * _inv[P % i] % P;
9             _invfac[i] = _inv[i] * _invfac[i - 1] % P;
10        }
11    }
12    i64 fac(int x) { return _fac[x]; }
13    i64 invfac(int x) { return _invfac[x]; }
14    i64 inv(int x) { return _inv[x]; }
15    i64 operator()(int n, int m) {
16        if(n < m || n < 0 || m < 0) return 0;
17        return _fac[n] * _invfac[m] % P * _invfac[n - m] % P;
18    }
19    std::vector<i64> _fac, _inv, _invfac; //阶乘, i的逆元, 阶乘的逆元
20 } comb(MAXN);

```

1.3.2 卢卡斯定理

$$C_n^m \pmod{p} = C_{\lfloor \frac{n}{p} \rfloor}^{\lfloor \frac{m}{p} \rfloor} * C_{n \bmod p}^{m \bmod p}$$

```
1 #include <bits/stdc++.h>
```

```

2 using i64 = long long;
3 constexpr i64 P = 19491001;
4 constexpr i64 MAXN = 19491001;
5
6 struct Comb {
7     Comb(int n) : _fac(n + 1, 1), _inv(n + 1, 1), _invfac(n + 1, 1) {
8         for(int i = 2; i <= n; ++i) {
9             _fac[i] = _fac[i - 1] * i % P;
10            _inv[i] = (P - P / i) * _inv[P % i] % P;
11            _invfac[i] = _inv[i] * _invfac[i - 1] % P;
12        }
13    }
14    i64 fac(int x) { return _fac[x]; }
15    i64 invfac(int x) { return _invfac[x]; }
16    i64 inv(int x) { return _inv[x]; }
17    i64 operator()(int n, int m) {
18        if(n < m || n < 0 || m < 0) return 0;
19        return _fac[n] * _invfac[m] % P * _invfac[n - m] % P;
20    }
21    std::vector<i64> _fac, _inv, _invfac; //阶乘, i的逆元, 阶乘的逆元
22 } comb(MAXN);
23
24 i64 lucas(i64 n, i64 m) {
25     if(m == 0) return 1;
26     return lucas(n / P, m / P) * comb(n % P, m % P) % P;
27 }
28
29 void solve() {
30     i64 n, m;
31     std::cin >> n >> m;
32     std::cout << lucas(n + m, m) << '\n';
33 }
34
35 //lucas定理, 求大数组合数
36 //https://www.luogu.com.cn/problem/P5160
37 int main() {
38     std::ios::sync_with_stdio(false);
39     std::cin.tie(nullptr);
40     int T = 1;
41     std::cin >> T;
42     while(T--) {
43         solve();
44     }
45     return 0;
46 }

```

1.4 多项式

1.4.1 FFT

模板题: Luogu P3803 模板题: ABC 392G

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 constexpr double PI = std::numbers::pi_v<double>;
5
6 void FFT(std::vector<std::complex<double>> &A, int opt = 1) {
7     int n = A.size();
8     std::vector<int> p(n);
9     for(int i = 0; i < n; ++i) {
10         p[i] = p[i / 2] / 2 + (n / 2) * (i & 1);
11     }
12     for(int i = 0; i < n; ++i) {
13         if(i < p[i]) {
14             std::swap(A[i], A[p[i]]);
15         }
16     }
17     for(int len = 2; len <= n; len <= 1) {
18         std::complex<double> w1 = {cos(2 * PI / len), sin(2 * PI / len) * opt};
19         for(int i = 0; i < n; i += len) {
20             std::complex<double> wk = {1, 0};
21             for(int j = 0; j < len / 2; ++j) {
22                 std::complex<double> x = A[i + j];
23                 std::complex<double> y = A[i + j + len / 2] * wk;
24                 A[i + j] = x + y;
25                 A[i + j + len / 2] = x - y;
26                 wk *= w1;
27             }
28         }
29     }
30 }
31
32 template<typename T>
33 std::vector<T> multiply(const std::vector<T> &A, const std::vector<T> &B) {
34     int n = std::bit_ceil(A.size() + B.size() - 1);
35     std::vector<std::complex<double>> v(n);
36     for(int i = 0; i < A.size(); ++i) {
37         v[i].real(A[i]);
38     }
39     for(int i = 0; i < B.size(); ++i) {
40         v[i].imag(B[i]);
41     }
42     v.resize(n);
43     FFT(v);

```

```

44     for(int i = 0; i < n; ++i) {
45         v[i] *= v[i];
46     }
47     FFT(v, -1);
48     std::vector<T> res(A.size() + B.size() - 1);
49     for(int i = 0; i < res.size(); ++i) {
50         res[i] = (T)round(v[i].imag() / 2 / n);
51     }
52     return res;
53 }
54
55 template<typename T>
56 std::vector<T> convolution(const std::vector<T> &A, std::vector<T> kernel) {
57     std::reverse(kernel.begin(), kernel.end());
58     auto res = multiply(A, kernel);
59     return std::vector(res.begin() + kernel.size() - 1, res.begin() + A.size());
60 }
61
62 int main() {
63     std::ios::sync_with_stdio(false);
64     std::cin.tie(nullptr);
65     int n, m;
66     std::cin >> n >> m;
67     std::vector<int> a(n + 1), b(m + 1);
68     for(int i = 0; i <= n; ++i) {
69         std::cin >> a[i];
70     }
71     for(int i = 0; i <= m; ++i) {
72         std::cin >> b[i];
73     }
74     auto c = multiply(a, b);
75     for(int i = 0; i < c.size(); ++i) {
76         std::cout << c[i] << " \n"[i + 1 == c.size()];
77     }
78     return 0;
79 }

```

1.4.2 NTT

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 constexpr i64 P = 998244353;
5 constexpr i64 G = 3;
6 constexpr i64 INVG = 332748118;
7
8 i64 qpow(i64 a, i64 b) {
9     i64 res = 1;

```

```

10     while(b) {
11         if(b & 1) {
12             res = res * a % P;
13         }
14         b >>= 1;
15         a = a * a % P;
16     }
17     return res;
18 }
19
20 void NTT(std::vector<i64> &A, int opt = 1) {
21     int n = A.size();
22     std::vector<int> p(n);
23     for(int i = 0; i < n; ++i) {
24         p[i] = p[i / 2] / 2 + (n / 2) * (i & 1);
25     }
26     for(int i = 0; i < n; ++i) {
27         if(i < p[i]) {
28             std::swap(A[i], A[p[i]]);
29         }
30     }
31     for(int len = 2; len <= n; len <= 1) {
32         i64 g1 = qpow(opt == 1 ? G : INVG, (P - 1) / len);
33         for(int i = 0; i < n; i += len) {
34             i64 gk = 1;
35             for(int j = 0; j < len / 2; ++j) {
36                 i64 x = A[i + j];
37                 i64 y = A[i + j + len / 2] * gk % P;
38                 A[i + j] = (x + y) % P;
39                 A[i + j + len / 2] = (x - y + P) % P;
40                 gk = (gk * g1) % P;
41             }
42         }
43     }
44 }
45
46 std::vector<i64> multiply(std::vector<i64> A, std::vector<i64> B) {
47     auto k = A.size() + B.size() - 1;
48     int n = std::bit_ceil(k);
49     A.resize(n), B.resize(n);
50     NTT(A), NTT(B);
51     for (int i = 0; i < n; ++i) {
52         A[i] = A[i] * B[i] % P;
53     }
54     NTT(A, -1);
55     i64 invn = qpow(n, P - 2);
56     A.resize(k);
57     for (auto &x : A) {

```

```

58     x = x * invn % P;
59 }
60 return A;
61 }
62
63 int main() {
64     std::ios::sync_with_stdio(false);
65     std::cin.tie(nullptr);
66     int n, m;
67     std::cin >> n >> m;
68     std::vector<i64> a(n + 1), b(m + 1);
69     for(int i = 0; i <= n; ++i) {
70         std::cin >> a[i];
71     }
72     for(int i = 0; i <= m; ++i) {
73         std::cin >> b[i];
74     }
75     auto c = multiply(a, b);
76     for(int i = 0; i < c.size(); ++i) {
77         std::cout << c[i] << " \n"[i + 1 == c.size()];
78     }
79     return 0;
80 }

```

1.4.3 Poly

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 constexpr i64 P = 998244353;
5 constexpr i64 G = 3;
6
7 i64 qpow(i64 a, i64 b) {
8     i64 res = 1;
9     while(b) {
10         if(b & 1) {
11             res = res * a % P;
12         }
13         b >>= 1;
14         a = a * a % P;
15     }
16     return res;
17 }
18
19 struct Poly : std::vector<i64> {
20     using std::vector<i64>::vector;
21     friend Poly& operator+=(Poly &A, const Poly &B) {
22         A.resize(std::max(A.size(), B.size()));

```

```

23         for(int i = 0; i < B.size(); ++i) {
24             A[i] = (A[i] + B[i]) % P;
25         }
26         return A;
27     }
28     friend Poly& operator-=(Poly &A, const Poly &B) {
29         A.resize(std::max(A.size(), B.size()));
30         for(int i = 0; i < B.size(); ++i) {
31             A[i] = (A[i] - B[i] + P) % P;
32         }
33         return A;
34     }
35     friend Poly& operator*=(Poly &A, Poly B) {
36         auto k = A.size() + B.size() - 1;
37         auto n = std::bit_ceil(k);
38         if(1LL * A.size() * B.size() <= 3 * n * std::bit_width(n)) {
39             Poly res(k);
40             for(int i = 0; i < A.size(); ++i) {
41                 for(int j = 0; j < B.size(); ++j) {
42                     res[i + j] = (res[i + j] + A[i] * B[j]) % P;
43                 }
44             }
45             A = std::move(res);
46         } else {
47             A.resize(n), B.resize(n);
48             NTT(A), NTT(B);
49             for(int i = 0; i < n; ++i) {
50                 A[i] = A[i] * B[i] % P;
51             }
52             NTT(A, -1);
53             std::reverse(A.begin() + 1, A.end());
54             i64 invn = qpow(n, P - 2);
55             A.resize(k);
56             for(auto &x : A) {
57                 x = x * invn % P;
58             }
59         }
60         return A;
61     }
62     friend Poly& operator/=(Poly &A, Poly B) {
63         if(A.size() < B.size()) return A = Poly{0};
64         int n = A.size() - B.size() + 1;
65         std::reverse(A.begin(), A.end());
66         std::reverse(B.begin(), B.end());
67         A = A % n * inv(B, n) % n;
68         std::reverse(A.begin(), A.end());
69         return A;
70     }

```

```

71 friend Poly& operator%=(Poly &A, const Poly &B) {
72     A = (A - A / B * B) % (B.size() - 1);
73     return A;
74 }
75 friend Poly& operator%=(Poly &A, size_t k) {
76     A.resize(std::min(A.size(), k));
77     return A;
78 }
79 friend Poly operator+(Poly A, const Poly &B) { return A += B; }
80 friend Poly operator-(Poly A, const Poly &B) { return A -= B; }
81 friend Poly operator*(Poly A, const Poly &B) { return A *= B; }
82 friend Poly operator/(Poly A, const Poly &B) { return A /= B; }
83 friend Poly operator%(Poly A, const Poly &B) { return A %= B; }
84 friend Poly operator%(Poly A, size_t k) { return A %= k; }
85
86 static Poly derive(const Poly &A) {
87     if(A.size() <= 1) return Poly{0};
88     Poly res(A.size() - 1);
89     for(int i = 1; i < A.size(); ++i) {
90         res[i - 1] = A[i] * i % P;
91     }
92     return res;
93 }
94 static Poly integral(const Poly &A) {
95     Poly res(A.size() + 1);
96     res[1] = A[0];
97     std::vector<i64> _inv(A.size() + 1, 1);
98     for(int i = 2; i <= A.size(); ++i) {
99         _inv[i] = (P - P / i) * _inv[P % i] % P;
100         res[i] = A[i - 1] * _inv[i] % P;
101     }
102     return res;
103 }
104 static Poly ln(const Poly &A, int n) {
105     return integral(derive(A) * inv(A, n) % n) % n;
106 }
107 static Poly pow(const Poly &A, i64 k, int n) {
108     return exp(Poly{k} * ln(A, n) % n, n) % n;
109 }
110 static Poly inv(const Poly &A, int n) {
111     Poly res = {qpow(A[0], P - 2)};
112     for(int k = 2; k < 2 * n; k <= 1) {
113         res = res * (Poly{2} - A % k * res) % k;
114     }
115     return res % n;
116 }
117 static Poly exp(const Poly &A, int n) {
118     Poly res = {1};

```

```

119     for(int k = 2; k < 2 * n; k <= 1) {
120         res = res * (Poly{1} - ln(res, k) + A % k) % k;
121     }
122     return res % n;
123 }
124 static Poly sqrt(const Poly &A, int n) {
125     Poly res = {1};
126     for(int k = 2; k < 2 * n; k <= 1) {
127         res = (res + A % k * inv(res, k) % k) * Poly{(P + 1) / 2} % k;
128     }
129     return res % n;
130 }
131 static Poly ln(const Poly &A) { return ln(A, A.size()); }
132 static Poly inv(const Poly &A) { return inv(A, A.size()); }
133 static Poly exp(const Poly &A) { return exp(A, A.size()); }
134 static Poly sqrt(const Poly &A) { return sqrt(A, A.size()); }
135 static Poly pow(const Poly &A, i64 k) { return pow(A, k, A.size()); }
136
137 Poly shift(int k) {
138     auto A = *this;
139     if (k >= 0) {
140         A.insert(A.begin(), k, 0);
141         return A;
142     } else if (A.size() <= -k) {
143         return Poly();
144     } else {
145         return Poly(A.begin() + (-k), A.end());
146     }
147 }
148 static void NTT(std::vector<i64> &A, int opt = 1) {
149     int n = A.size();
150     for(int i = 0, j = 0; i < n; ++i) {
151         if(i > j) {
152             std::swap(A[i], A[j]);
153         }
154         for(int k = n >> 1; (j ^ k) < k; k >>= 1);
155     }
156     init(n);
157     for(int i = 1; i < n; i <= 1) {
158         for(int j = 0; j < n; j += (i << 1)) {
159             for(int k = j; k < i + j; ++k) {
160                 int t = w[k + i - j] * A[i + k] % P;
161                 A[k + i] = (A[k] - t < 0 ? A[k] - t + P : A[k] - t);
162                 A[k] = (A[k] + t >= P ? A[k] + t - P : A[k] + t);
163             }
164         }
165     }
166 }

```

```

167 static void init(int n) {
168     int m = w.size();
169     if (n <= m) return;
170     w.resize(n);
171     i64 wn = qpow(G, (P - 1) / n);
172     w[n >> 1] = 1;
173     for (int i = (n >> 1) + 1; i < n; ++i) {
174         w[i] = w[i - 1] * wn % P;
175     }
176     for (int i = n - 1; i >= m; --i) {
177         w[i >> 1] = w[i];
178     }
179 }
180 inline static std::vector<i64> w{1};
181 };
182
183 int main() {
184     std::ios::sync_with_stdio(false);
185     std::cin.tie(nullptr);
186     Poly A{1, 2, 3};
187
188     return 0;
189 }

```

1.5 取模类

```

1 constexpr i64 P = 998244353;
2 template<typename T>
3 T qpow(T a, i64 b) {
4     T res = 1;
5     while(b) {
6         if(b & 1) {
7             res = res * a;
8         }
9         b >>= 1;
10        a = a * a;
11    }
12    return res;
13 }
14
15 struct M64 {
16     M64() = default;
17     M64(const i64 a) : x((a % P + P) % P) {}
18     i64 val() const { return x; }
19     M64 inv() const { return qpow(*this, P - 2); }
20 #define OP(op, EXPR) \
21     friend M64& operator op##= (M64 &A, const M64 &B) { \
22         return A = M64(EXPR); \

```

```

23     } \
24     friend M64 operator op (M64 A, const M64 &B) { \
25         return A op##= B; \
26     }
27     OP(+, A.val() + B.val())
28     OP(-, A.val() - B.val())
29     OP(*, A.val() * B.val())
30     OP(/, A.val() * B.inv().val())
31     OP(&, A.val() & B.val())
32     OP(|, A.val() | B.val())
33     OP(^, A.val() ^ B.val())
34 #undef OP
35     friend M64& operator++(M64 &a) {
36         a += 1;
37         return a;
38     }
39     friend M64 operator++(M64 &a, int) {
40         M64 t = a;
41         a += 1;
42         return t;
43     }
44     friend std::ostream& operator<<(std::ostream &os, const M64 &m) {
45         return os << m.x;
46     }
47     friend std::istream& operator>>(std::istream &is, M64 &m) {
48         i64 tmp;
49         is >> tmp;
50         m = M64(tmp);
51         return is;
52     }
53     i64 x{};
54 };

```

2 图论

2.1 图的连通性

2.1.1 Tarjan 割点

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 //tarjan求割点
5 //https://www.luogu.com.cn/problem/P3388
6 int main() {
7     std::ios::sync_with_stdio(false);
8     std::cin.tie(nullptr);

```

```

9  int n, m;
10 std::cin >> n >> m;
11 std::vector<std::vector<int>>> v(n + 1);
12 for(int i = 1; i <= m; ++i) {
13     int x, y;
14     std::cin >> x >> y;
15     v[x].push_back(y);
16     v[y].push_back(x);
17 }
18 std::vector<int> dfn(n + 1), low(n + 1), cutPoint(n + 1);
19 int cnt = 0, root = 0;
20 auto dfs = [&](auto self, int id, int lst) ->void {
21     dfn[id] = low[id] = ++cnt;
22     int sz = 0; //儿子个数
23     for(auto nxt : v[id]) {
24         if(!dfn[nxt]) {
25             sz++;
26             self(self, nxt, id);
27             low[id] = std::min(low[id], low[nxt]);
28             if(low[nxt] >= dfn[id]) {
29                 cutPoint[id] = 1;
30             }
31         } else if(nxt != lst) {
32             low[id] = std::min(low[id], dfn[nxt]);
33         }
34     }
35     if(sz <= 1 && id == root) {
36         cutPoint[id] = 0;
37     }
38 };
39 for(int i = 1; i <= n; ++i) {
40     if(!dfn[i]) {
41         root = i;
42         dfs(dfs, i, 0);
43     }
44 }
45 std::cout << std::count(cutPoint.begin() + 1, cutPoint.end(), 1) << '\n';
46 for(int i = 1; i <= n; ++i) {
47     if(cutPoint[i] == 1) {
48         std::cout << i << ' ';
49     }
50 }
51 return 0;
52 }

```

2.1.2 Tarjan 割边

```
1 #include <bits/stdc++.h>
```

```

2 using i64 = long long;
3
4 //tarjan求割边
5 //https://www.luogu.com.cn/problem/P1656
6 int main() {
7     std::ios::sync_with_stdio(false);
8     std::cin.tie(nullptr);
9     int n, m;
10    std::cin >> n >> m;
11    std::vector<std::vector<std::pair<int, int>>> v(n + 1);
12    for(int i = 1; i <= m; ++i) {
13        int x, y;
14        std::cin >> x >> y;
15        v[x].push_back({y, i}); //记录边id(从1开始), 防止重边
16        v[y].push_back({x, i});
17    }
18    std::vector<int> dfn(n + 1), low(n + 1);
19    std::vector<std::pair<int, int>> bridge;
20    int cnt = 0;
21    auto dfs = [&](auto self, int id, int lid) ->void {
22        dfn[id] = low[id] = ++cnt;
23        for(auto [nxt, eid] : v[id]) {
24            if(!dfn[nxt]) {
25                self(self, nxt, eid);
26                low[id] = std::min(low[id], low[nxt]);
27                if(low[nxt] == dfn[nxt]) { //是割边
28                    bridge.push_back({id, nxt});
29                }
30            } else if(eid != lid) {
31                low[id] = std::min(low[id], dfn[nxt]);
32            }
33        }
34    };
35    for(int i = 1; i <= n; ++i) {
36        if(!dfn[i]) {
37            dfs(dfs, i, 0);
38        }
39    }
40    std::sort(bridge.begin(), bridge.end());
41    for(auto [x, y] : bridge) {
42        std::cout << x << ' ' << y << '\n';
43    }
44    return 0;
45 }

```

2.1.3 Tarjan 强连通分量

```
1 #include <bits/stdc++.h>
```

```

2 using i64 = long long;
3
4 //tarjan求强连通分量(scc)
5 //https://www.luogu.com.cn/problem/B3609
6 int main() {
7     std::ios::sync_with_stdio(false);
8     std::cin.tie(nullptr);
9     int n, m;
10    std::cin >> n >> m;
11    std::vector<std::vector<int>> v(n + 1);
12    for(int i = 0; i < m; ++i) {
13        int x, y;
14        std::cin >> x >> y;
15        v[x].push_back(y);
16    }
17    std::vector<std::vector<int>> scc(n + 1);
18    std::vector<int> dfn(n + 1), low(n + 1), ins(n + 1), bel(n + 1);
19    std::stack<int> stk;
20    int cnt = 0, tot = 0;
21    auto dfs = [&](auto self, int id) ->void {
22        dfn[id] = low[id] = ++cnt;
23        stk.push(id);
24        ins[id] = 1;
25        for(auto nxt : v[id]) {
26            if(!dfn[nxt]) {
27                self(self, nxt);
28                low[id] = std::min(low[id], low[nxt]);
29            } else if(ins[nxt]) {
30                low[id] = std::min(low[id], low[nxt]);
31            }
32        }
33        if(dfn[id] == low[id]) {
34            ++tot;
35            while(true) {
36                int num = stk.top();
37                stk.pop();
38                ins[num] = 0;
39                bel[num] = tot;
40                scc[tot].push_back(num);
41                if(id == num) break;
42            }
43        }
44    };
45    for(int i = 1; i <= n; ++i) {
46        if(!dfn[i]) {
47            dfs(dfs, i);
48        }
49    }

```

```

50     for(int i = 1; i <= tot; ++i) {
51         std::sort(scc[i].begin(), scc[i].end());
52     }
53     std::sort(scc.begin() + 1, scc.begin() + tot + 1);
54     std::cout << tot << '\n';
55     for(int i = 1; i <= tot; ++i) {
56         for(int j = 0; j < scc[i].size(); ++j) {
57             std::cout << scc[i][j] << " \n"[j == scc[i].size() - 1];
58         }
59     }
60     return 0;
61 }

```

2.1.4 Tarjan 点双连通分量

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 //tarjan求点双连通分量
5 //https://www.luogu.com.cn/problem/P8435
6 int main() {
7     std::ios::sync_with_stdio(false);
8     std::cin.tie(nullptr);
9     int n, m;
10    std::cin >> n >> m;
11    std::vector<std::vector<int>> v(n + 1);
12    for(int i = 1; i <= m; ++i) {
13        int x, y;
14        std::cin >> x >> y;
15        v[x].push_back(y);
16        v[y].push_back(x);
17    }
18    std::vector<std::vector<int>> vcc(n + 1);
19    std::vector<int> dfn(n + 1), low(n + 1);
20    std::stack<int> stk;
21    int cnt = 0, tot = 0;
22    auto dfs = [&](auto self, int id, int lst) ->void {
23        dfn[id] = low[id] = ++cnt;
24        stk.push(id);
25        int sz = 0;
26        for(auto nxt : v[id]) {
27            if(!dfn[nxt]) {
28                sz++;
29                self(self, nxt, id);
30                low[id] = std::min(low[id], low[nxt]);
31                if(low[nxt] >= dfn[id]) {
32                    ++tot;
33                    while(true) {

```

```

34         int num = stk.top();
35         stk.pop();
36         vcc[tot].push_back(num);
37         if(num == nxt) break;
38     }
39     vcc[tot].push_back(id);
40 }
41 } else if(nxt != lst) {
42     low[id] = std::min(low[id], dfn[nxt]);
43 }
44 }
45 if(lst == 0 && sz == 0) {
46     ++tot;
47     vcc[tot].push_back(id);
48 }
49 };
50 for(int i = 1; i <= n; ++i) {
51     if(!dfn[i]) {
52         dfs(dfs, i, 0);
53     }
54 }
55 std::cout << tot << '\n';
56 for(int i = 1; i <= tot; ++i) {
57     std::cout << vcc[i].size() << ' ';
58     for(int j = 0; j < vcc[i].size(); ++j) {
59         std::cout << vcc[i][j] << " \n"[j == vcc[i].size() - 1];
60     }
61 }
62 return 0;
63 }

```

2.1.5 Tarjan 边双连通分量

用途: 求边双连通分量

模板题: [洛谷 P8436](#)

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 int main() {
5     std::ios::sync_with_stdio(false);
6     std::cin.tie(nullptr);
7     int n, m;
8     std::cin >> n >> m;
9     std::vector<std::vector<std::pair<int, int>>> v(n + 1);
10    for(int i = 1; i <= m; ++i) {
11        int x, y;
12        std::cin >> x >> y;

```

```

13        v[x].push_back({y, i});
14        v[y].push_back({x, i});
15    }
16    std::vector<std::vector<int>> ecc(n + 1);
17    std::vector<int> dfn(n + 1), low(n + 1);
18    std::stack<int> stk;
19    int cnt = 0, tot = 0;
20    auto dfs = [&](auto self, int id, int lid) ->void {
21        dfn[id] = low[id] = ++cnt;
22        stk.push(id);
23        for(auto [nxt, eid] : v[id]) {
24            if(!dfn[nxt]) {
25                self(self, nxt, eid);
26                low[id] = std::min(low[id], low[nxt]);
27            } else if(lid != eid) {
28                low[id] = std::min(low[id], dfn[nxt]);
29            }
30        }
31        if(dfn[id] == low[id]) {
32            ++tot;
33            while(true) {
34                int num = stk.top();
35                ecc[tot].push_back(num);
36                stk.pop();
37                if(id == num) break;
38            }
39        }
40    };
41    for(int i = 1; i <= n; ++i) {
42        if(!dfn[i]) {
43            dfs(dfs, i, 0);
44        }
45    }
46    std::cout << tot << '\n';
47    for(int i = 1; i <= tot; ++i) {
48        std::cout << ecc[i].size() << ' ';
49        for(int j = 0; j < ecc[i].size(); ++j) {
50            std::cout << ecc[i][j] << " \n"[j == ecc[i].size() - 1];
51        }
52    }
53    return 0;
54 }

```

2.2 重链剖分

```

1 #include <bits/stdc++.h>
2
3 struct HLD {

```

```

4  HLD(const int &n) : n(_n), v(_n + 1) {
5      fa = dep = son = sz = top = in = out = rin = std::vector<int>(_n + 1);
6  }
7
8  void addEdge(const int &x, const int &y) {
9      v[x].push_back(y);
10     v[y].push_back(x);
11 }
12
13 void dfs1(int id, int &t) {
14     sz[id] = 1;
15     in[id] = t;
16     rin[t] = id;
17     for(const auto &nxt : v[id]) {
18         if(nxt == fa[id]) continue;
19         fa[nxt] = id;
20         dep[nxt] = dep[id] + 1;
21         dfs1(nxt, ++t);
22         sz[id] += sz[nxt];
23         if(sz[son[id]] < sz[nxt]) {
24             son[id] = nxt;
25         }
26     }
27     out[id] = t;
28 }
29 void dfs2(int id, int t) {
30     top[id] = t;
31     if(son[id] == 0) return;
32     dfs2(son[id], t);
33     for(const auto &nxt : v[id]) {
34         if(nxt == fa[id] || nxt == son[id]) continue;
35         dfs2(nxt, t);
36     }
37 }
38 void work(int root = 1) {
39     int dfsn = 1;
40     dfs1(root, dfsn);
41     dfs2(root, root);
42 }
43
44 bool isAncestor(int x, int y) {
45     return in[x] <= in[y] && out[x] >= out[y];
46 }
47
48 int lca(int x, int y) {
49     while(top[x] != top[y]) {
50         if(dep[top[x]] < dep[top[y]]) {
51             std::swap(x, y);

```

```

52         }
53         x = fa[top[x]];
54     }
55     return (dep[x] < dep[y] ? x : y);
56 }
57
58 int dis(int x, int y) {
59     return dep[x] + dep[y] - 2 * dep[lca(x, y)];
60 }
61
62 int kth(int id, int k) {
63     if(k > dep[id]) return 0;
64     while(dep[id] - dep[top[id]] + 1 <= k) {
65         k -= (dep[id] - dep[top[id]] + 1);
66         id = fa[top[id]];
67     }
68     return rin[in[id] - k];
69 }
70
71 std::vector<std::vector<int>> v;
72 std::vector<int> fa, dep, son, sz, top, in, out, rin;
73 int n;
74 };
75
76 //树链剖分求LCA
77 //https://www.luogu.com.cn/problem/P3379
78 int main() {
79     std::ios::sync_with_stdio(0);
80     std::cin.tie(nullptr);
81     int n, m, s;
82     std::cin >> n >> m >> s;
83     HLD tree(n);
84     for(int i = 0; i < n - 1; ++i) {
85         int x, y;
86         std::cin >> x >> y;
87         tree.addEdge(x, y);
88     }
89     tree.work(s);
90     for(int i = 0; i < m; ++i) {
91         int x, y;
92         std::cin >> x >> y;
93         std::cout << tree.lca(x, y) << '\n';
94     }
95     return 0;
96 }

```

2.3 长链剖分

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 struct LLD {
5     LLD(const int &n) : n(_n) {
6         son = len = dep = top = in = out = lg = std::vector<int>(n + 1);
7         v = up = down = std::vector<std::vector<int>>(n + 1);
8         m = std::bit_width(std::bit_ceil((unsigned)n));
9         fa = std::vector<vector<int>>(m + 1, std::vector<int>(n + 1));
10        for(int i = 2; i <= n; ++i) {
11            lg[i] = lg[i >> 1] + 1;
12        }
13    }
14
15    void addEdge(const int &x, const int &y) {
16        v[x].push_back(y);
17        v[y].push_back(x);
18    }
19
20    void dfs1(int id, int &t) {
21        for(int i = 0; i < m; ++i) {
22            fa[i + 1][id] = fa[i][fa[i][id]];
23        }
24        in[id] = t;
25        len[id] = 1;
26        for(const auto &nxt : v[id]) {
27            if(nxt == fa[0][id]) continue;
28            fa[0][nxt] = id;
29            dep[nxt] = dep[id] + 1;
30            dfs1(nxt, ++t);
31            if(len[nxt] + 1 > len[id]) {
32                len[id] = len[nxt] + 1;
33                son[id] = nxt;
34            }
35        }
36        out[id] = t;
37    }
38
39    void dfs2(int id, int t) {
40        top[id] = t;
41        if(son[id] == 0) return;
42        dfs2(son[id], t);
43        for(const auto &nxt : v[id]) {
44            if(nxt == fa[0][id] || nxt == son[id]) continue;
45            dfs2(nxt, t);
46        }
47    }
48
49    void work(int root = 1) {
50        int dfsn = 1;
51        dfs1(root, dfsn);
52        dfs2(root, root);
53        for(int i = 1; i <= n; ++i) {
54            if(top[i] != i) continue;
55            for(int j = 1, now = i; j <= len[i] && now; ++j, now = fa[0][now]) {
56                up[i].push_back(now);
57            }
58            for(int j = 1, now = i; j <= len[i] && now; ++j, now = son[now]) {
59                down[i].push_back(now);
60            }
61        }
62    }
63
64    bool isAncestor(int x, int y) { //x是y的祖先
65        return in[x] <= in[y] && out[x] >= out[y];
66    }
67
68    int lca(int x, int y) {
69        while(top[x] != top[y]) {
70            if(dep[top[x]] < dep[top[y]]) {
71                std::swap(x, y);
72            }
73            x = fa[0][top[x]];
74        }
75        return (dep[x] < dep[y] ? x : y);
76    }
77
78    int dis(int x, int y) {
79        return dep[x] + dep[y] - 2 * dep[lca(x, y)];
80    }
81
82    int kth(int id, int k) {
83        if(k == 0) return id;
84        int t = lg[k];
85        k -= (1 << t);
86        id = fa[t][id];
87        int p = top[id];
88        if(dep[id] - dep[p] >= k) {
89            id = down[p][(dep[id] - dep[p]) - k];
90        } else {
91            id = up[p][k - (dep[id] - dep[p])];
92        }
93        return id;
94    }
95

```

```

96     std::vector<std::vector<int>> v, up, down, fa;
97     std::vector<int> son, len, dep, top, in, out, lg;
98     int n, m;
99 };
100
101 int main() {
102     std::ios::sync_with_stdio(false);
103     std::cin.tie(nullptr);
104     int n, m, s;
105     std::cin >> n >> m >> s;
106     LLD tree(n);
107     for(int i = 1; i <= n - 1; ++i) {
108         int x, y;
109         std::cin >> x >> y;
110         tree.addEdge(x, y);
111     }
112     tree.work(s);
113     for(int i = 1; i <= m; ++i) {
114         int x, y;
115         std::cin >> x >> y;
116         std::cout << tree.lca(x, y) << '\n';
117     }
118     return 0;
119 }

```

2.4 树的重心

定义：如果在树中选择某个节点并删除，这棵树将分为若干棵子树，统计子树节点数并记录最大值。取遍树上所有节点，使此最大值取到最小的节点被称为整个树的重心。

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 void solve() {
5     int n;
6     std::cin >> n;
7     std::vector<std::vector<int>> v(n + 1);
8     for(int i = 1; i <= n - 1; ++i) {
9         int x, y;
10        std::cin >> x >> y;
11        v[x].push_back(y);
12        v[y].push_back(x);
13    }
14    std::vector<int> sz(n + 1), weight(n + 1);
15    int ans = n;
16    auto dfs = [&](auto self, int id, int lst) ->void {
17        sz[id] = 1;
18        for(auto nxt : v[id]) {

```

```

19            if(nxt == lst) continue;
20            self(self, nxt, id);
21            weight[id] = std::max(weight[id], sz[nxt]);
22            sz[id] += sz[nxt];
23        }
24        weight[id] = std::max(weight[id], n - sz[id]);
25        ans = std::min(ans, weight[id]);
26    };
27    dfs(dfs, 1, 0);
28    for(int i = 1; i <= n; ++i) {
29        if(weight[i] == ans) {
30            std::cout << i << ' ';
31            break;
32        }
33    }
34    std::cout << ans << '\n';
35 }
36 //树的重心（重心最多有两个）
37 //http://bailian.openjudge.cn/practice/1655/
38 int main() {
39     std::ios::sync_with_stdio(false);
40     std::cin.tie(nullptr);
41     int T = 1;
42     std::cin >> T;
43     while(T--) {
44         solve();
45     }
46     return 0;
47 }

```

2.5 欧拉回路

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 // 无向图欧拉回路or通路
4 struct Euler {
5     int id = 0;
6     vector<int> edg; // 存储边
7     vector<vector<int>> graph; // 存储图
8     // 初始化
9     Euler(int n, int m) {
10         graph.resize(n + 1);
11         edg.resize(m + 1);
12     }
13     // 添加边
14     void add_edge(int u, int v) {
15         id++;
16         graph[u].push_back(id);

```

```

17     graph[v].push_back(id);
18     edg[id] = u ^ v;
19 }
20 // 判断是否存在欧拉通路and返回起点
21 int is_semiEuler() {
22     int n = graph.size() - 1;
23     int odd = 0, start = 0;
24     for (int i = 1; i <= n; i++) {
25         if (graph[i].size() & 1) {
26             odd++;
27             start = i;
28         }
29     }
30     if (odd == 0) return 1;
31     if (odd == 2) return start;
32     return 0;
33 }
34 // 判断是否存在欧拉回路
35 bool is_Euler() {
36     int n = graph.size() - 1;
37     for (int i = 1; i <= n; i++) {
38         if (graph[i].size() & 1) {
39             return false;
40         }
41     }
42     return true;
43 }
44 // 求解欧拉回路or通路
45 vector<int> Euler_tour(int start) {
46     vector<int> tour;
47     function<void(int)> dfs = [&](int u) {
48         while (!graph[u].empty()) {
49             int i = graph[u].back();
50             graph[u].pop_back();
51             if (edg[i] == -1) continue;
52             int v = edg[i] ^ u;
53             edg[i] = -1;
54             dfs(v);
55         }
56         tour.push_back(u);
57     };
58     dfs(start);
59     return tour;
60 }
61 };
62 // 有向图欧拉回路or通路
63 struct Directed_Euler {
64     vector<int> inE, outE;    // 存储入,出度

```

```

65     vector<vector<int>> graph; // 存储图
66     int n, m;
67     // 初始化
68     Directed_Euler(int n, int m) : n(n), m(m) {
69         graph.resize(n + 1);
70         inE.resize(n + 1);
71         outE.resize(n + 1);
72     }
73     // 添加边
74     void add_edge(int u, int v) {
75         graph[u].push_back(v);
76         outE[u]++;
77         inE[v]++;
78     }
79     // 判断是否存在欧拉通路and返回起点
80     int is_semiEuler() {
81         int odd = 0, neodd = 0, start = 0;
82         for (int i = 1; i <= n; i++) {
83             if (outE[i] - inE[i] == 1) {
84                 odd++;
85                 start = i;
86             } else if (inE[i] - outE[i] == 1) {
87                 neodd++;
88             } else if (inE[i] != outE[i]) {
89                 return 0;
90             }
91         }
92         if (odd == 1 && neodd == 1) {
93             return start;
94         }
95         if (odd == 0 && neodd == 0) {
96             return 1;
97         }
98         return 0;
99     }
100     // 判断是否存在欧拉回路
101     bool is_Euler() {
102         int n = graph.size() - 1;
103         for (int i = 1; i <= n; i++) {
104             if (inE[i] != outE[i])
105                 return false;
106         }
107         return true;
108     }
109     // 求解欧拉回路or通路
110     vector<int> Euler_tour(int start) {
111         vector<int> tour;
112         function<void(int)> dfs = [&](int u) {

```

```

113         while (!graph[u].empty()) {
114             int v = graph[u].back();
115             graph[u].pop_back();
116             dfs(v);
117         }
118         tour.push_back(u);
119     };
120     dfs(start);
121     return tour; // 返回的是逆序的欧拉回路or通路
122 }
123 };

```

2.6 流和匹配

2.6.1 EdmondsKarp

时间复杂度: $O(|V||E|^2)$ 实际情况一般远低于此复杂度

空间复杂度: $O(|V| + |E|)$

用途: 求最大流

模板题: 洛谷 P3376

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template<typename T>
5 struct MaxFolw {
6     struct Edge {
7         Edge() = default;
8         Edge(int _nxt, T _cap, int _enxt) : nxt(_nxt), cap(_cap), enxt(_enxt) {}
9         int nxt, enxt;
10        T cap;
11    };
12    MaxFolw(int n) : head(n, -1), pre(n), mf(n) {}
13    void addEdge(int x, int y, T cap) {
14        edge.push_back(Edge(y, cap, head[x]));
15        head[x] = edge.size() - 1;
16        edge.push_back(Edge(x, 0, head[y]));
17        head[y] = edge.size() - 1;
18    }
19    bool bfs(int s, int t) {
20        std::fill(mf.begin(), mf.end(), 0);
21        mf[s] = INF;
22        std::queue<int> q;
23        q.push(s);
24        while(!q.empty()) {
25            int id = q.front();
26            q.pop();
27            for(int eid = head[id]; ~eid; eid = edge[eid].enxt) {

```

```

28                auto &[nxt, _, cap] = edge[eid];
29                if(mf[nxt] == 0 && cap > 0) {
30                    mf[nxt] = std::min(mf[id], cap);
31                    pre[nxt] = eid;
32                    if(nxt == t) return true;
33                    q.push(nxt);
34                }
35            }
36        }
37        return false;
38    };
39    T flow(int s, int t) {
40        T flow = 0;
41        while(bfs(s, t)) { //找到增广路
42            for(int id = t; id != s; id = edge[pre[id] ^ 1].nxt) {
43                edge[pre[id]].cap -= mf[t];
44                edge[pre[id] ^ 1].cap += mf[t];
45            }
46            flow += mf[t];
47        }
48        return flow;
49    }
50    std::vector<Edge> edge;
51    std::vector<int> head, pre; // pre: id的前驱边
52    std::vector<T> mf; //每S~v的流量上限,
53    const T INF = INT_MAX;
54 };
55
56 int main() {
57     std::ios::sync_with_stdio(false);
58     std::cin.tie(nullptr);
59     int n, m, S, T;
60     std::cin >> n >> m >> S >> T;
61     MaxFolw<i64> mf(n + 1);
62     for(int i = 0; i < m; ++i) {
63         int x, y, cap;
64         std::cin >> x >> y >> cap;
65         mf.addEdge(x, y, cap);
66     }
67     std::cout << mf.flow(S, T) << '\n';
68     return 0;
69 }

```

2.6.2 二分图判定

时间复杂度: $O(|V| + |E|)$

空间复杂度: $O(|V|)$

模板题: Luogu P1330

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 int main() {
5     std::ios::sync_with_stdio(false);
6     std::cin.tie(nullptr);
7     int n, m;
8     std::cin >> n >> m;
9     std::vector<std::vector<int>>> v(n + 1);
10    for(int i = 1; i <= m; ++i) {
11        int x, y;
12        std::cin >> x >> y;
13        v[x].push_back(y);
14        v[y].push_back(x);
15    }
16    std::vector<int> col(n + 1), vis(n + 1); //染色值1/2, 是否标记
17    auto dfs = [&](auto self, int id, int val) ->bool { //判定是否是二分图
18        col[id] = val;
19        vis[id] = 1;
20        for(auto nxt : v[id]) {
21            if(!col[nxt]) {
22                if(!self(self, nxt, val ^ 3)) {
23                    return false;
24                }
25            } else if(col[nxt] == val) {
26                return false;
27            }
28        }
29        return true;
30    };
31    int ans = 0;
32    for(int i = 1; i <= n; ++i) {
33        if(!vis[i]) {
34            col = std::vector<int>(n + 1);
35            if(!dfs(dfs, i, 1)) {
36                std::cout << "Impossible\n";
37                exit(0);
38            }
39            int A = std::count(col.begin(), col.end(), 1);
40            int B = std::count(col.begin(), col.end(), 2);
41            ans += std::min(A, B);
42        }
43    }
44    std::cout << ans << '\n';
45    return 0;
46 }

```

2.6.3 二分图最大匹配

时间复杂度: $O(|V_1||V_2|)$

空间复杂度: $O(|E| + |V_1| + |V_2|)$

模板题: [Luogu P3386](#)

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 struct BipartiteGraph {
5     BipartiteGraph(int n, int m)
6         : n(n), m(m), g(n + 1), vis(m + 1), mch(m + 1) {};
7     void add(int x, int y) {
8         g[x].push_back(y);
9     }
10    bool dfs(int id) {
11        for(auto nxt : g[id]) {
12            if(!vis[nxt]) {
13                vis[nxt] = 1;
14                if(!mch[nxt] || dfs(mch[nxt])) {
15                    mch[nxt] = id;
16                    return true;
17                }
18            }
19        }
20        return false;
21    }
22    int solve() { //求最大匹配
23        int res = 0;
24        for(int i = 1; i <= n; ++i) {
25            std::fill(vis.begin(), vis.end(), false);
26            res += dfs(i);
27        }
28        return res;
29    }
30    int n, m;
31    std::vector<std::vector<int>>> g; //存图
32    std::vector<bool> vis; //标记是否搜索过
33    std::vector<int> mch; //mch[i]表示i号点匹配的编号
34 };
35
36 int main() {
37     std::ios::sync_with_stdio(false);
38     std::cin.tie(nullptr);
39     int n, m, k;
40     std::cin >> n >> m >> k;
41     BipartiteGraph bg(n + 1, m + 1);
42     for(int i = 1; i <= k; ++i) {
43         int x, y;

```

```

44     std::cin >> x >> y;
45     bg.add(x, y);
46 }
47 std::cout << bg.solve() << '\n';
48 return 0;
49 }

```

3 数据结构

3.1 树状数组

3.1.1 树状数组

```

1 #include<bits/stdc++.h>
2
3 //树状数组(Fenwick)
4 //https://www.luogu.com.cn/problem/P3374
5 template<typename T>
6 struct Fenwick {
7     Fenwick(int n) : v(n + 1) {};
8     void update(int x, T dx) {
9         for(int i = x; i < v.size(); i += (i & -i)) {
10             v[i] += dx;
11         }
12     }
13     T query(int x) {
14         T res{};
15         for(int i = x; i > 0; i -= (i & -i)) {
16             res += v[i];
17         }
18         return res;
19     }
20     T range(int l, int r) {
21         return query(r) - query(l - 1);
22     }
23     T kth(T k) {
24         int pos = 0;
25         int logn = std::bit_width(v.size() - 1);
26         for(int i = 1 << (logn - 1); i > 0; i >= 1) {
27             if(pos + i < v.size() && v[pos + i] < k) {
28                 k -= v[pos + i];
29                 pos += i;
30             }
31         }
32         return pos + 1;
33     }
34     std::vector<T> v;

```

```

35 };
36
37 int main() {
38     std::ios::sync_with_stdio(false);
39     std::cin.tie(nullptr);
40     int n, m;
41     std::cin >> n >> m;
42     Fenwick<int> tr(n);
43     for(int i = 1; i <= n; ++i) {
44         int x;
45         std::cin >> x;
46         tr.update(i, x);
47     }
48     for(int i = 0; i < m; ++i) {
49         int o, x, y;
50         std::cin >> o >> x >> y;
51         if(o == 1) {
52             tr.update(x, y);
53         } else if (o == 2) {
54             std::cout << tr.range(x, y) << '\n';
55         }
56     }
57     return 0;
58 };

```

3.1.2 树状数组 2

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template<typename T>
5 struct Fenwick {
6     Fenwick(int n) : vec(n + 1), add(n + 1) {}
7     void rangeUpdate(int l, int r, T dx) {
8         updatedx(l, dx);
9         updatedx(r + 1, -dx);
10    }
11    void update(int pos, T dx) {
12        rangeUpdate(pos, pos, dx);
13    }
14    T rangeQuery(int l, int r) {
15        return query(r) - query(l - 1);
16    }
17    T query(int pos) {
18        T res{};
19        for(int i = pos; i >= 1; i -= (i & -i)) {
20            res += pos * vec[i] - add[i];
21        }

```

```

22     return res;
23 }
24 void updatedx(int pos, T dx) {
25     for(int i = pos; i < vec.size(); i += (i & -i)) {
26         vec[i] += dx;
27         add[i] += (pos - 1) * dx;
28     }
29 }
30 std::vector<T> vec, add;
31 };
32
33 //树状数组，区间修改，区间查询
34 //https://www.luogu.com.cn/problem/P3372
35 int main() {
36     std::ios::sync_with_stdio(false);
37     std::cin.tie(nullptr);
38     int n, m;
39     std::cin >> n >> m;
40     Fenwick<i64> tr(n);
41     for(int i = 1; i <= n; ++i) {
42         int x;
43         std::cin >> x;
44         tr.rangeUpdate(i, i, x);
45     }
46     for(int i = 1; i <= m; ++i) {
47         int opt;
48         std::cin >> opt;
49         if(opt == 1) {
50             int l, r, dx;
51             std::cin >> l >> r >> dx;
52             tr.rangeUpdate(l, r, dx);
53         } else if(opt == 2) {
54             int l, r;
55             std::cin >> l >> r;
56             std::cout << tr.rangeQuery(l, r) << '\n';
57         }
58     }
59     return 0;
60 }

```

3.2 线段树

3.2.1 线段树 simple

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template<typename Info>

```

```

5 struct SegmentTree {
6     #define ls (id<<1)
7     #define rs (id<<1|1)
8     SegmentTree() = default;
9     SegmentTree(int n) : n(n), info(n << 2) {}
10    SegmentTree(const SegmentTree<Info> &o) : n(o.n), info(o.info) {}
11    SegmentTree(const std::vector<Info> &init) : SegmentTree((int)init.size()) {
12        auto build = [&](auto self, int id, int l, int r) ->void {
13            if(l == r) {
14                info[id] = init[l];
15                return;
16            }
17            int mid = (l + r) / 2;
18            self(self, ls, l, mid);
19            self(self, rs, mid + 1, r);
20            pushup(id);
21        };
22        build(build, 1, 0, n - 1);
23    }
24    void pushup(int id) {
25        info[id] = info[ls] + info[rs];
26    }
27    void update(int pos, const Info &val) {
28        update(1, 0, n - 1, pos, val);
29    }
30    Info query(int pos) {
31        return rangeQuery(pos, pos);
32    }
33    Info rangeQuery(int l, int r) {
34        return rangeQuery(1, 0, n - 1, l, r);
35    }
36    void update(int id, int l, int r, int pos, const Info &val) {
37        if(l == r) {
38            info[id] = val;
39            return;
40        }
41        int mid = (l + r) / 2;
42        if(pos <= mid) {
43            update(ls, l, mid, pos, val);
44        } else {
45            update(rs, mid + 1, r, pos, val);
46        }
47        pushup(id);
48    }
49    Info rangeQuery(int id, int l, int r, int x, int y) {
50        if(x <= l && r <= y) {
51            return info[id];
52        }

```

```

53     int mid = (l + r) / 2;
54     Info res;
55     if(x <= mid) {
56         res = res + rangeQuery(ls, l, mid, x, y);
57     }
58     if(y > mid) {
59         res = res + rangeQuery(rs, mid + 1, r, x, y);
60     }
61     return res;
62 }
63 #undef ls
64 #undef rs
65 int n;
66 std::vector<Info> info;
67 };
68
69 constexpr int INF = 2E9;
70
71 struct Info {
72     Info() = default;
73     Info(int x, int idx) {
74         lmn = rmx = x - idx;
75         lmx = rmn = x + idx;
76     }
77     int lmn = INF;
78     int rmn = -INF;
79     int lmx = INF;
80     int rmx = -INF;
81     int ans = 0;
82 };
83
84 Info operator+(const Info &x, const Info &y) {
85     Info res;
86     res.lmx = std::max(x.lmx, y.lmx);
87     res.rmx = std::max(x.rmx, y.rmx);
88     res.lmn = std::min(x.lmn, y.lmn);
89     res.rmn = std::min(x.rmn, y.rmn);
90     res.ans = std::max({x.ans, y.ans, x.lmx - y.rmn, y.rmx - x.lmn});
91     return res;
92 }
93
94 void solve() {
95     int n, q;
96     std::cin >> n >> q;
97     std::vector<Info> v(n + 1);
98     for(int i = 1; i <= n; ++i) {
99         int x;
100         std::cin >> x;

```

```

101         v[i] = Info(x, i);
102     }
103     SegmentTree<Info> tr(v);
104     std::cout << tr.rangeQuery(1, n).ans << '\n';
105     for(int i = 1; i <= q; ++i) {
106         int idx, x;
107         std::cin >> idx >> x;
108         tr.update(idx, Info(x, idx));
109         std::cout << tr.rangeQuery(1, n).ans << '\n';
110     }
111 }
112
113 int main() {
114     std::ios::sync_with_stdio(false);
115     std::cin.tie(nullptr);
116     int T = 1;
117     std::cin >> T;
118     while(T--) {
119         solve();
120     }
121     return 0;
122 }

```

3.2.2 线段树

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 //线段树，区间修改，区间查询
5 //https://www.luogu.com.cn/problem/P3372
6 template<typename Info, typename Tag>
7 struct SegmentTree {
8     #define ls (id<<1)
9     #define rs (id<<1|1)
10     SegmentTree() = default;
11     SegmentTree(int n) : SegmentTree(std::vector<Info>(n)) {}
12     SegmentTree(int n, const Info &val) : SegmentTree(std::vector<Info>(n, val)) {}
13     SegmentTree(const std::vector<Info> &init) : n(init.size()), info(n << 2), tag(n
        << 2), len(n << 2) {
14         auto build = [&](auto self, int id, int l, int r) ->void {
15             len[id] = r - l + 1;
16             if(l == r) {
17                 info[id] = init[l];
18                 return;
19             }
20             int mid = (l + r) / 2;
21             self(self, ls, l, mid);
22             self(self, rs, mid + 1, r);

```

```

23     pushup(id);
24 };
25 build(build, 1, 0, n - 1);
26 }
27 void apply(int id, const Tag &dx) {
28     info[id].apply(dx, len[id]);
29     tag[id].apply(dx);
30 }
31 void pushup(int id) {
32     info[id] = info[ls] + info[rs];
33 }
34 void pushdown(int id) {
35     apply(ls, tag[id]);
36     apply(rs, tag[id]);
37     tag[id] = Tag();
38 }
39 void rangeUpdate(int l, int r, const Tag &dx) {
40     rangeUpdate(1, 0, n - 1, l, r, dx);
41 }
42 void update(int pos, const Tag &dx) {
43     rangeUpdate(pos, pos, dx);
44 }
45 Info rangeQuery(int l, int r) {
46     return rangeQuery(1, 0, n - 1, l, r);
47 }
48 Info query(int pos) {
49     return rangeQuery(pos, pos);
50 }
51 void rangeUpdate(int id, int l, int r, int x, int y, const Tag &dx) {
52     if(x <= l && r <= y) {
53         apply(id, dx);
54         return;
55     }
56     int mid = (l + r) / 2;
57     pushdown(id);
58     if(x <= mid) {
59         rangeUpdate(ls, l, mid, x, y, dx);
60     }
61     if(y > mid) {
62         rangeUpdate(rs, mid + 1, r, x, y, dx);
63     }
64     pushup(id);
65 }
66 Info rangeQuery(int id, int l, int r, int x, int y) {
67     if(x <= l && r <= y) {
68         return info[id];
69     }
70     int mid = (l + r) / 2;

```

```

71     pushdown(id);
72     Info res = Info::unit();
73     if(x <= mid) {
74         res = res + rangeQuery(ls, l, mid, x, y);
75     }
76     if(y > mid) {
77         res = res + rangeQuery(rs, mid + 1, r, x, y);
78     }
79     return res;
80 }
81 int findFirst(int x, int y, auto check) {
82     Info pre = Info::unit();
83     return findFirst(1, 0, n - 1, x, y, pre, check);
84 }
85 int findFirst(int id, int l, int r, int x, int y, Info& pre, auto check) {
86     if(x <= l && r <= y) {
87         if(!check(pre + info[id])) {
88             pre = pre + info[id];
89             return -1;
90         } else if(l == r) {
91             return l;
92         }
93     }
94     int mid = (l + r) / 2;
95     pushdown(id);
96     if(x <= mid) {
97         int res = findFirst(ls, l, mid, x, y, pre, check);
98         if(res != -1) return res;
99     }
100     return findFirst(rs, mid + 1, r, x, y, pre, check);
101 }
102 #undef ls
103 #undef rs
104 int n;
105 std::vector<Info> info;
106 std::vector<Tag> tag;
107 std::vector<int> len;
108 };
109
110 constexpr i64 INF = 4E18;
111 i64 P = 571373;
112
113 struct Tag {
114     i64 add = 0;
115     i64 mul = 1;
116     void apply(const Tag &dx) {
117         mul = (mul * dx.mul) % P;
118         add = (add * dx.mul + dx.add) % P;

```

```

119     }
120 };
121
122 struct Info {
123     Info() = default;
124     Info(i64 x) : sum(x % P) {}
125     i64 sum = 0;
126     void apply(const Tag &dx, const int &len) {
127         sum = (sum * dx.mul + dx.add * len) % P;
128     }
129     static Info unit() {
130         return Info();
131     }
132 };
133
134 Info operator+(const Info &x, const Info &y) {
135     Info res;
136     res.sum = (x.sum + y.sum) % P;
137     return res;
138 }
139
140 int main() {
141     std::ios::sync_with_stdio(false);
142     std::cin.tie(nullptr);
143     int n, m;
144     std::cin >> n >> m >> P;
145     std::vector<Info> v(n);
146     for(int i = 0; i < n; ++i) {
147         i64 x;
148         std::cin >> x;
149         v[i] = x;
150     }
151     SegmentTree<Info, Tag> tr(v);
152     while(m--) {
153         i64 opt, x, y;
154         std::cin >> opt >> x >> y;
155         x--, y--;
156         if(opt == 1) {
157             i64 k;
158             std::cin >> k;
159             tr.rangeUpdate(x, y, Tag(0, k));
160         } else if(opt == 2) {
161             i64 k;
162             std::cin >> k;
163             tr.rangeUpdate(x, y, Tag(k, 1));
164         } else if(opt == 3) {
165             std::cout << tr.rangeQuery(x, y).sum << '\n';
166         }
167     }

```

```

167     }
168     return 0;
169 }

```

3.2.3 线段树优化建图

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 struct STOG {
5     #define ls (id<<1)
6     #define rs (id<<1|1)
7     STOG(int n) : n(n), in(n << 2), out(n << 2), v(n * 7) {
8         int tot = n;
9         auto build = [&](auto self, int id, int l, int r) ->void {
10             if(l == r) {
11                 in[id] = out[id] = l;
12                 return;
13             }
14             int mid = (l + r) / 2;
15             self(self, ls, l, mid);
16             self(self, rs, mid + 1, r);
17             in[id] = ++tot;
18             out[id] = ++tot;
19             update(in[id], in[ls], 0);
20             update(in[id], in[rs], 0);
21             update(out[ls], out[id], 0);
22             update(out[rs], out[id], 0);
23         };
24         build(build, 1, 1, n);
25     }
26     void update(int x, int y, int w) { //连一条从x 到 y的边, 边权为w
27         v[x].emplace_back(y, w);
28     }
29     //model == 0 时, 从pos 到 [x, y]连边, 边权为w
30     //model == 1 时, 从[x, y] 到 pos连边, 边权为w
31     void rangeUpdate(int pos, int x, int y, int w, int model) {
32         rangeUpdate(1, 1, n, pos, x, y, w, model);
33     }
34     void rangeUpdate(int id, int l, int r, int pos, int x, int y, int w, auto model)
35     {
36         if(x <= l && r <= y) {
37             if(model == 0) {
38                 update(pos, in[id], w);
39             } else {
40                 update(out[id], pos, w);
41             }
42             return;
43         }

```

```

42     }
43     int mid = (l + r) / 2;
44     if(x <= mid) {
45         rangeUpdate(ls, l, mid, pos, x, y, w, model);
46     }
47     if(y > mid) {
48         rangeUpdate(rs, mid + 1, r, pos, x, y, w, model);
49     }
50 }
51 #undef ls
52 #undef rs
53 int n;
54 std::vector<int> in, out;
55 std::vector<std::vector<std::pair<int, int>>> v;
56 };
57
58 int main() {
59     std::ios::sync_with_stdio(false);
60     std::cin.tie(nullptr);
61     int n, q, s;
62     std::cin >> n >> q >> s;
63     STOG tr(n);
64     for(int i = 1; i <= q; ++i) {
65         int opt;
66         std::cin >> opt;
67         if(opt == 1) {
68             int pos, x, w;
69             std::cin >> pos >> x >> w;
70             tr.update(pos, x, w);
71         } else if(opt == 2) {
72             int pos, x, y, w;
73             std::cin >> pos >> x >> y >> w;
74             tr.rangeUpdate(pos, x, y, w, 0);
75         } else if(opt == 3) {
76             int pos, x, y, w;
77             std::cin >> pos >> x >> y >> w;
78             tr.rangeUpdate(pos, x, y, w, 1);
79         }
80     }
81     auto &graph = tr.v;
82     int m = tr.v.size() - 1;
83     std::vector<i64> dp(m + 1, LLONG_MAX);
84     std::priority_queue<std::pair<i64, int>, std::vector<std::pair<i64, int>>, std::greater<>> pq;
85     pq.emplace(0LL, s);
86     while(!pq.empty()) {
87         auto [w, id] = pq.top();
88         pq.pop();

```

```

89         if(w >= dp[id]) continue;
90         dp[id] = w;
91         for(const auto &[nxt, dx] : graph[id]) {
92             i64 ww = w + dx;
93             if(ww < dp[nxt]) {
94                 pq.emplace(ww, nxt);
95             }
96         }
97     }
98     for(int i = 1; i <= n; ++i) {
99         std::cout << (dp[i] == LLONG_MAX ? -1 : dp[i]) << " \n"[i == n];
100     }
101     return 0;
102 }

```

3.2.4 动态开点线段树

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3 constexpr int MAXN = 2E5;
4
5 template<typename Info, typename Tag, typename T = i64>
6 struct SegmentTree {
7     #define ls (id->l)
8     #define rs (id->r)
9     struct Node {
10         Node* l = nullptr;
11         Node* r = nullptr;
12         Tag tag;
13         Info info;
14     };
15     SegmentTree() = default;
16     SegmentTree(T n) : L(0), R(n - 1) {}
17     SegmentTree(T L, T R) : L(L), R(R) {}
18     void apply(Node* &id, const Tag &dx, T len) {
19         if(id == nullptr) id = new Node();
20         id->info.apply(dx, len);
21         id->tag.apply(dx);
22     }
23     void pushdown(Node* &id, T l, T r) {
24         if(id->tag == Tag()) return;
25         T mid = l + r >> 1;
26         apply(ls, id->tag, mid - l + 1);
27         apply(rs, id->tag, r - mid);
28         id->tag = Tag();
29     }
30     #define pushdown(id) pushdown(id, l, r)
31     void pushup(Node* id) {

```

```

32     id->info = (ls == nullptr ? Info::unit() : ls->info)
33         + (rs == nullptr ? Info::unit() : rs->info);
34 }
35 void update(T pos, const Tag &dx) {
36     rangeUpdate(pos, pos, dx);
37 }
38 void rangeUpdate(T x, T y, const Tag &dx) {
39     rangeUpdate(root, L, R, x, y, dx);
40 }
41 void rangeUpdate(Node* &id, T l, T r, T x, T y, const Tag &dx) {
42     if(id == nullptr) id = new Node();
43     if(x <= l && r <= y) {
44         apply(id, dx, r - l + 1);
45         return;
46     }
47     T mid = l + r >> 1;
48     pushdown(id);
49     if(x <= mid) {
50         rangeUpdate(ls, l, mid, x, y, dx);
51     }
52     if(y > mid) {
53         rangeUpdate(rs, mid + 1, r, x, y, dx);
54     }
55     pushup(id);
56 }
57 Info query(T pos) {
58     return rangeQuery(pos, pos);
59 }
60 Info rangeQuery(T l, T r) {
61     return rangeQuery(root, L, R, l, r);
62 }
63 Info rangeQuery(Node* &id, T l, T r, T x, T y) {
64     if(id == nullptr) return Info::unit();
65     if(x <= l && r <= y) {
66         return id->info;
67     }
68     T mid = l + r >> 1;
69     pushdown(id);
70     Info res = Info::unit();
71     if(x <= mid) {
72         res = res + rangeQuery(ls, l, mid, x, y);
73     }
74     if(y > mid) {
75         res = res + rangeQuery(rs, mid + 1, r, x, y);
76     }
77     return res;
78 }
79 void merge(SegmentTree<Info, Tag, T> seg) {

```

```

80     root = merge(root, seg.root, L, R);
81 }
82 Node* merge(Node* &xid, Node* &yid, T l, T r) {
83     if(xid == nullptr) return yid;
84     if(yid == nullptr) return xid;
85     if(l == r) {
86         xid->info = (xid->info ^ yid->info);
87         delete yid;
88         return xid;
89     }
90     T mid = l + r >> 1;
91     pushdown(xid);
92     pushdown(yid);
93     xid->l = merge(xid->l, yid->l, l, mid);
94     xid->r = merge(xid->r, yid->r, mid + 1, r);
95     pushup(xid);
96     delete yid;
97     return xid;
98 }
99 SegmentTree<Info, Tag, T> split(T l, T r) { //分裂出[l, r]的部分
100     SegmentTree<Info, Tag, T> seg(L, R);
101     seg.root = split(root, L, R, l, r);
102     return seg;
103 }
104 Node* split(Node* &id, T l, T r, T x, T y) {
105     if(id == nullptr || r < x || l > y) return nullptr;
106     Node* nid = nullptr;
107     if(x <= l && r <= y) {
108         std::swap(id, nid);
109         return nid;
110     }
111     nid = new Node();
112     T mid = l + r >> 1;
113     pushdown(id);
114     if(x <= mid) {
115         nid->l = split(ls, l, mid, x, y);
116     }
117     if(y > mid) {
118         nid->r = split(rs, mid + 1, r, x, y);
119     }
120     pushup(id);
121     pushup(nid);
122     return nid;
123 }
124 T findFirst(T x, T y, auto check) {
125     Info pre = Info::unit();
126     return findFirst(root, L, R, x, y, pre, check);
127 }

```

```

128 T findFirst(Node* &id, T l, T r, T x, T y, Info& pre, auto check) {
129     if(id == nullptr) return -1;
130     if(x <= l && r <= y) {
131         if(!check(pre + id->info)) {
132             pre = pre + id->info;
133             return -1;
134         } else if(l == r) {
135             return l;
136         }
137     }
138     T mid = l + r >> 1;
139     pushdown(id);
140     if(x <= mid) {
141         int res = findFirst(ls, l, mid, x, y, pre, check);
142         if(res != -1) return res;
143     }
144     return findFirst(rs, mid + 1, r, x, y, pre, check);
145 }
146 #undef ls
147 #undef rs
148 #undef pushdown
149 T L, R;
150 Node* root = nullptr;
151 };
152
153 struct Tag {
154     Tag() = default;
155     Tag(int _add) : add(_add) {}
156     void apply(const Tag &dx) {
157         add += dx.add;
158     }
159     bool operator==(const Tag&) const = default;
160     int add = 0;
161 };
162
163 struct Info {
164     Info() = default;
165     Info(i64 _val) : sum(_val) {}
166     template<typename T>
167     void apply(const Tag &dx, const T &len) {
168         sum += dx.add * len;
169     }
170     static Info unit() {
171         return Info();
172     }
173     i64 sum = 0;
174 };
175

```

```

176 Info operator+(const Info &x, const Info &y) {
177     Info res;
178     res.sum = x.sum + y.sum;
179     return res;
180 }
181
182 Info operator^(const Info &x, const Info &y) {
183     Info res;
184     res.sum = x.sum + y.sum;
185     return res;
186 }
187
188 int main() {
189     std::ios::sync_with_stdio(false);
190     std::cin.tie(nullptr);
191     int n, m, idx = 1;
192     std::cin >> n >> m;
193     std::vector segs(n + 1, SegmentTree<Info, Tag, int>(MAXN + 1));
194     for(int i = 1; i <= n; ++i) {
195         int x;
196         std::cin >> x;
197         segs[idx].update(i, x);
198     }
199     for(int i = 0; i < m; ++i) {
200         int opt;
201         std::cin >> opt;
202         if(opt == 0) {
203             int p, x, y;
204             std::cin >> p >> x >> y;
205             segs[++idx] = segs[p].split(x, y);
206         } else if(opt == 1) {
207             int p, t;
208             std::cin >> p >> t;
209             segs[p].merge(segs[t]);
210         } else if(opt == 2) {
211             int p, x, q;
212             std::cin >> p >> x >> q;
213             segs[p].update(q, x);
214         } else if(opt == 3) {
215             int p, x, y;
216             std::cin >> p >> x >> y;
217             std::cout << segs[p].rangeQuery(x, y).sum << '\n';
218         } else if(opt == 4) {
219             int p, k;
220             std::cin >> p >> k;
221             std::cout << segs[p].findFirst(1, n, [&](const auto& info) ->bool {
222                 return info.sum >= k;
223             }) << '\n';

```

```

224     }
225 }
226 return 0;
227 }

```

3.2.5 主席树

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template<typename Info, typename Tag>
5 struct PersistentTree {
6     struct Node {
7         int l = 0, r = 0;
8         Info info;
9     };
10    #define ls(x) (node[x].l)
11    #define rs(x) (node[x].r)
12    PersistentTree(int n) : PersistentTree(std::vector<Info>(n + 1)) {}
13    PersistentTree(const std::vector<Info> &init) : n((int)init.size() - 1) {
14        node.reserve(n << 3);
15        auto build = [&](auto self, int l, int r) ->int {
16            node.push_back(Node());
17            int id = node.size() - 1;
18            if(l == r) {
19                node[id].info = init[l];
20            } else {
21                int mid = (l + r) / 2;
22                ls(id) = self(self, l, mid);
23                rs(id) = self(self, mid + 1, r);
24                node[id].info = node[ls(id)].info + node[rs(id)].info;
25            }
26            return id;
27        };
28        root.push_back(build(build, 1, n));
29    };
30    int update(int version, int pos, const Info &val) {
31        root.push_back(update(root[version], 1, n, pos, val));
32        return root.size() - 1;
33    }
34    int update(int version, int pos, const Tag &dx) {
35        root.push_back(update(root[version], 1, n, pos, dx));
36        return root.size() - 1;
37    }
38    Info query(int version, int pos) {
39        return rangeQuery(version, pos, pos);
40    }
41    Info rangeQuery(int version, int l, int r) {

```

```

42        return rangeQuery(root[version], 1, n, l, r);
43    }
44    int update(int lst, int l, int r, const int &pos, const Info &val) {
45        node.push_back(node[lst]);
46        int id = node.size() - 1;
47        if(l == r) {
48            node[id].info = val;
49        } else {
50            int mid = (l + r) / 2;
51            if(pos <= mid) {
52                ls(id) = update(ls(lst), l, mid, pos, val);
53            } else if(pos > mid) {
54                rs(id) = update(rs(lst), mid + 1, r, pos, val);
55            }
56            node[id].info = node[ls(id)].info + node[rs(id)].info;
57        }
58        return id;
59    }
60    int update(int lst, int l, int r, const int &pos, const Tag &dx) {
61        node.push_back(node[lst]);
62        int id = node.size() - 1;
63        if(l == r) {
64            node[id].info.apply(dx);
65        } else {
66            int mid = (l + r) / 2;
67            if(pos <= mid) {
68                ls(id) = update(ls(lst), l, mid, pos, dx);
69            } else if(pos > mid) {
70                rs(id) = update(rs(lst), mid + 1, r, pos, dx);
71            }
72            node[id].info = node[ls(id)].info + node[rs(id)].info;
73        }
74        return id;
75    }
76    Info rangeQuery(int id, int l, int r, const int &x, const int &y) {
77        if(x <= l && r <= y) {
78            return node[id].info;
79        }
80        int mid = (l + r) / 2;
81        Info res;
82        if(x <= mid) {
83            res = res + rangeQuery(ls(id), l, mid, x, y);
84        }
85        if(y > mid) {
86            res = res + rangeQuery(rs(id), mid + 1, r, x, y);
87        }
88        return res;
89    }

```

```

90 int kth(int versionl, int versionr, int k) {
91     return kth(root[versionl], root[versionr], 1, n, k);
92 }
93 int kth(int idx, int idy, int l, int r, int k) { //静态区间第k小, 不支持修改
94     if(l >= r) return l;
95     int mid = (l + r) / 2;
96     int dx = node[ls(idy)].info.sum - node[ls(idx)].info.sum;
97     if(dx >= k) {
98         return kth(ls(idx), ls(idy), l, mid, k);
99     } else {
100         return kth(rs(idx), rs(idy), mid + 1, r, k - dx);
101     }
102 }
103 #undef ls
104 #undef rs
105 const int n;
106 std::vector<Node> node;
107 std::vector<int> root;
108 };
109
110 struct Tag {
111     Tag() = default;
112     Tag(int dx = 0) : add(dx) {}
113     int add = 0;
114     void apply(const Tag &dx) {
115         add += dx.add;
116     }
117 };
118
119 struct Info {
120     int sum = 0;
121     void apply(const Tag &dx) {
122         sum += dx.add;
123     }
124 };
125
126 Info operator+(const Info &x, const Info &y) {
127     Info res;
128     res.sum = x.sum + y.sum;
129     return res;
130 }
131 //主席树(单点修改, 历史版本区间查询, 静态区间第k小)
132 //https://www.luogu.com.cn/problem/P3834
133 int main() {
134     std::ios::sync_with_stdio(false);
135     std::cin.tie(nullptr);
136     int n, q;
137     std::cin >> n >> q;

```

```

138     std::vector<int> v(n + 1), tmp(n + 1);
139     for(int i = 1; i <= n; ++i) {
140         std::cin >> v[i];
141         tmp[i] = v[i];
142     }
143     std::sort(tmp.begin() + 1, tmp.end());
144     tmp.erase(std::unique(tmp.begin() + 1, tmp.end()), tmp.end());
145     int m = tmp.size() - 1;
146     PersistentTree<Info, Tag> tr(std::vector<Info>(m + 1));
147     std::vector<int> version(n + 1);
148     version[0] = tr.root.size() - 1;
149     for(int i = 1; i <= n; ++i) {
150         int pos = std::lower_bound(tmp.begin() + 1, tmp.end(), v[i]) - tmp.begin();
151         version[i] = tr.update(version[i - 1], pos, Tag(1));
152     }
153     for(int i = 1; i <= q; ++i) {
154         int l, r, k;
155         std::cin >> l >> r >> k;
156         int pos = tr.kth(version[l - 1], version[r], k);
157         std::cout << tmp[pos] << '\n';
158     }
159     return 0;
160 }

```

3.2.6 标记永久化主席树

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template<typename Info, typename Tag>
5 struct PersistentTree {
6     struct Node {
7         int l = 0, r = 0;
8         Info info;
9         Tag tag;
10     };
11 #define ls(x) (node[id].l)
12 #define rs(x) (node[id].r)
13     PersistentTree(int n) : n(n) {}
14     PersistentTree(const std::vector<Info> &init) : PersistentTree((int)init.size()
15         - 1) {
16         node.reserve(n << 3);
17         auto build = [&](auto self, int l, int r) ->int {
18             node.push_back(Node());
19             int id = node.size() - 1;
20             if(l == r) {
21                 node[id].info = init[l];
22             } else {

```

```

22         int mid = (l + r) / 2;
23         ls(id) = self(self, l, mid);
24         rs(id) = self(self, mid + 1, r);
25         node[id].info = node[ls(id)].info + node[rs(id)].info;
26     }
27     return id;
28 };
29 root.push_back(build(build, 1, n));
30 };
31 int update(int version, int t, const Tag &dx) {
32     return rangeUpdate(version, t, t, dx);
33 }
34 Info query(int version, int t) {
35     return rangeQuery(version, t, t);
36 }
37 int rangeUpdate(int version, int l, int r, const Tag &dx) {
38     root.push_back(rangeUpdate(root[version], 1, n, l, r, dx));
39     return root.size() - 1;
40 }
41 Info rangeQuery(int version, int l, int r) {
42     return rangeQuery(root[version], 1, n, l, r);
43 }
44 int rangeUpdate(int lst, int l, int r, const int &x, const int &y, const Tag &dx
45 ) {
46     node.push_back(node[lst]);
47     int id = node.size() - 1;
48     node[id].info.apply(std::min(r, y) - std::max(l, x) + 1, dx);
49     if(x <= l && r <= y) {
50         node[id].tag.apply(dx);
51     } else {
52         int mid = (l + r) / 2;
53         if(x <= mid) {
54             ls(id) = rangeUpdate(ls(lst), l, mid, x, y, dx);
55         }
56         if(y > mid) {
57             rs(id) = rangeUpdate(rs(lst), mid + 1, r, x, y, dx);
58         }
59     }
60     return id;
61 }
62 Info rangeQuery(int id, int l, int r, const int &x, const int &y) {
63     if(x <= l && r <= y) {
64         return node[id].info;
65     }
66     int mid = (l + r) / 2;
67     Info res;
68     if(x <= mid) {
69         res = res + rangeQuery(ls(id), l, mid, x, y);

```

```

69     }
70     if(y > mid) {
71         res = res + rangeQuery(rs(id), mid + 1, r, x, y);
72     }
73     res.apply(std::min(r, y) - std::max(l, x) + 1, node[id].tag);
74     return res;
75 }
76 #undef ls
77 #undef rs
78 const int n;
79 std::vector<Node> node;
80 std::vector<int> root;
81 };
82
83 struct Tag {
84     Tag(int dx = 0) : add(dx) {}
85     int add = 0;
86     void apply(const Tag &dx) {
87         add += dx.add;
88     }
89 };
90
91 struct Info {
92     int sum = 0;
93     void apply(int len, const Tag &dx) {
94         sum += 1LL * len * dx.add;
95     }
96 };
97
98 Info operator+(const Info &x, const Info &y) {
99     Info res;
100     res.sum = x.sum + y.sum;
101     return res;
102 }
103
104 //可持久化线段树(区间修改, 区间历史查询)
105 //https://www.luogu.com.cn/problem/P3919
106 int main() {
107     std::ios::sync_with_stdio(false);
108     std::cin.tie(nullptr);
109     int n, q;
110     std::cin >> n >> q;
111     std::vector<Info> v(n + 1);
112     for(int i = 1; i <= n; ++i) {
113         std::cin >> v[i].sum;
114     }
115     PersistentTree<Info, Tag> tr(v);
116     std::vector<int> version(q + 1);

```

```

117 for(int i = 1; i <= q; ++i) {
118     int ver, opt, pos;
119     std::cin >> ver >> opt >> pos;
120     if(opt == 1) {
121         int x;
122         std::cin >> x;
123         int lst = tr.query(version[ver], pos).sum;
124         version[i] = tr.update(version[ver], pos, Tag(x - lst));
125     } else if(opt == 2) {
126         std::cout << tr.query(version[ver], pos).sum << '\n';
127         version[i] = version[ver];
128     }
129 }
130 return 0;
131 }

```

3.3 ST 表

时间复杂度:

- 初始化: $O(n \log(n))$
- 查询: $O(1)$

空间复杂度: $O(n \log(n))$

用途: RMQ 问题, 不支持修改

模板题: [Luogu P3865](#)

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template <typename T, typename Func = std::function<T(const T&, const T&)>>
5 struct ST {
6     ST() = default;
7     ST(const std::vector<T> &v, Func func = [](const T& a, const T& b) {
8         return std::max(a, b);
9     }) : func(std::move(func)) {
10         int k = std::__lg(v.size());
11         st = std::vector<std::vector<T>>(k + 1, std::vector<T>(v.size()));
12         st[0] = v;
13         for(int i = 0; i < k; ++i) {
14             for(int j = 0; j + (1 << (i + 1)) - 1 < v.size(); ++j) {
15                 st[i + 1][j] = this->func(st[i][j], st[i][j + (1 << i)]);
16             }
17         }
18     }
19     T range(int l, int r) {
20         int t = std::__lg(r - l + 1);
21         return func(st[t][l], st[t][r + 1 - (1 << t)]);
22     }
23 }

```

```

23 std::vector<std::vector<T>> st;
24 Func func;
25 };
26
27 //ST表(sparseTable)
28 //https://www.luogu.com.cn/problem/P3865
29 int main() {
30     std::ios::sync_with_stdio(false);
31     std::cin.tie(nullptr);
32     int n, q;
33     std::cin >> n >> q;
34     std::vector<int> v(n + 1);
35     for(int i = 1; i <= n; ++i) {
36         std::cin >> v[i];
37     }
38     ST<int> st(v);
39     while(q--) {
40         int l, r;
41         std::cin >> l >> r;
42         std::cout << st.range(l, r) << '\n';
43     }
44     return 0;
45 }

```

3.4 并查集

```

1 #include <bits/stdc++.h>
2
3 //并查集(disjoint set union)
4 //https://www.luogu.com.cn/problem/P3367
5 struct DSU {
6     DSU() = default;
7     DSU(int n) : p(n), sz(n, 1) {
8         std::iota(p.begin(), p.end(), 0);
9     }
10     int find(int x) {
11         return p[x] == x ? x : p[x] = find(p[x]);
12     }
13     bool same(int x, int y) {
14         return find(x) == find(y);
15     }
16     int merge(int x, int y) {
17         if (same(x, y)) return 0;
18         x = find(x), y = find(y);
19         if (sz[x] < sz[y]) std::swap(x, y);
20         sz[x] += sz[y];
21         p[y] = x;
22         return x;
23     }
24 }

```

```

23     }
24     int& size(int x) {
25         return sz[find(x)];
26     }
27     std::vector<int> p, sz;
28 };
29
30 int main() {
31     std::ios::sync_with_stdio(false);
32     std::cin.tie(nullptr);
33     int n, m;
34     std::cin >> n >> m;
35     DSU dsu(n);
36     for(int i = 0; i < m; ++i) {
37         int z, x, y;
38         std::cin >> z >> x >> y;
39         if(z == 1) {
40             dsu.merge(x, y);
41         } else if(z == 2) {
42             std::cout << (dsu.same(x, y) ? 'Y' : 'N') << '\n';
43         }
44     }
45     return 0;
46 }

```

3.5 可撤销并查集

```

1 #include <bits/stdc++.h>
2
3 struct RDSU {
4     RDSU() = default;
5     RDSU(int n) : p(n), sz(n, 1) {
6         std::iota(p.begin(), p.end(), 0);
7     }
8     int find(int x) {
9         while(p[x] != x) x = p[x];
10        return x;
11    }
12    bool same(int x, int y) {
13        return find(x) == find(y);
14    }
15    int merge(int x, int y) {
16        if (same(x, y)) return 0;
17        x = find(x), y = find(y);
18        if (sz[x] < sz[y]) std::swap(x, y);
19        hsz.push({sz[x], sz[y]});
20        hfa.push({p[y], p[y]});
21        sz[x] += sz[y];

```

```

22        p[y] = x;
23        return x;
24    }
25    int& size(int x) {
26        return sz[find(x)];
27    }
28    size_t now() {
29        return hsz.size();
30    }
31    void version(int ver) {
32        rollback(now() - ver);
33    }
34    void rollback(int t = 1) {
35        for(int i = 1; i <= t && !hfa.empty(); ++i) {
36            hfa.top().first = hfa.top().second;
37            hsz.top().first = hsz.top().second;
38            hfa.pop(), hsz.pop();
39        }
40    }
41    std::vector<int> p, sz;
42    std::stack<std::pair<int&, int>> hsz, hfa;
43 };
44
45 //https://www.starrycoding.com/problem/9
46 int main() {
47     std::ios::sync_with_stdio(false);
48     std::cin.tie(nullptr);
49     int n, q;
50     std::cin >> n >> q;
51     RDSU rdsu(n);
52     for(int i = 1; i <= q; ++i) {
53         int opt;
54         std::cin >> opt;
55         if(opt == 1) {
56             int x, y;
57             std::cin >> x >> y;
58             rdsu.merge(x, y);
59         } else if(opt == 2) {
60             rdsu.rollback();
61         } else if(opt == 3) {
62             int x, y;
63             std::cin >> x >> y;
64             std::cout << (rdsu.same(x, y) ? "YES" : "NO") << '\n';
65         }
66     }
67     return 0;
68 }

```

3.6 带权并查集

模板题: Luogu P1196

```
1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 struct DSUT {
5     DSUT() = default;
6     DSUT(int n) : fa(n), f(n), sz(n, 1) {
7         std::iota(fa.begin(), fa.end(), 0);
8     }
9     int find(int id) {
10         if(id == fa[id]) return id;
11         int root = find(fa[id]);
12         f[id] += f[fa[id]];
13         return fa[id] = root;
14     }
15     void merge(int x, int y) { //要依据题意修改
16         int fx = find(x), fy = find(y);
17         fa[fx] = fy;
18         f[fx] += sz[fy];
19         sz[fy] += sz[fx];
20     }
21     bool query(int x, int y) {
22         return find(x) == find(y);
23     }
24     void set(int pos, int val) {
25         f[pos] = val;
26     }
27     int get(int pos) {
28         return f[pos];
29     }
30     std::vector<int> fa, f, sz; //父节点, 到父节点的权值, 集合大小
31 };
32
33 int main() {
34     std::ios::sync_with_stdio(false);
35     std::cin.tie(nullptr);
36     int n;
37     std::cin >> n;
38     DSUT dsut(n);
39     for(int i = 1; i <= n; ++i) {
40         char opt;
41         int x, y;
42         std::cin >> opt >> x >> y;
43         if(opt == 'M') {
44             dsut.merge(x, y);
45         } else if(opt == 'C') {
```

```
46             if(!dsut.query(x, y)) {
47                 std::cout << -1 << '\n';
48             } else {
49                 std::cout << std::abs(dsut.get(x) - dsut.get(y)) - 1 << '\n';
50             }
51         }
52     }
53     return 0;
54 }
```

3.7 左偏树

```
1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template<typename T, typename _Compare = std::less<T>>
5 struct LeftistHeap {
6     struct Node {
7         Node(const T &info) : info(_info){}
8         T info;
9         int dis = -1;
10         Node *ls = nullptr;
11         Node *rs = nullptr;
12     };
13     LeftistHeap() = default;
14     LeftistHeap(_Compare _cmp) : cmp(std::move(_cmp)) {}
15     void merge(LeftistHeap &o) {
16         _size += o.size();
17         root = merge(root, o.root);
18     }
19     int dis(Node* &node) {
20         return node == nullptr ? -1 : node->dis;
21     }
22     Node* merge(Node* &x, Node* &y) {
23         if(x == nullptr) return y;
24         if(y == nullptr) return x;
25         if(cmp(x->info, y->info)) std::swap(x, y);
26         x->rs = merge(x->rs, y);
27         if(dis(x->ls) < dis(x->rs)) {
28             std::swap(x->ls, x->rs);
29         }
30         x->dis = dis(x->rs) + 1;
31         return x;
32     }
33     void push(const T &info) {
34         root = merge(root, new Node(info));
35         ++_size;
36     }
```

```

37 void pop() {
38     Node* temp = root;
39     root = merge(root->ls, root->rs);
40     delete temp;
41     --_size;
42 }
43 T top() {
44     assert(root != nullptr);
45     return root->info;
46 }
47 size_t size() {
48     return _size;
49 }
50 bool empty() {
51     return _size == 0;
52 }
53 _Compare cmp;
54 Node* root = nullptr;
55 size_t _size = 0;
56 };
57
58 int main() {
59     std::ios::sync_with_stdio(false);
60     std::cin.tie(nullptr);
61     return 0;
62 }

```

3.8 智慧集

时间复杂度:

- 插入: $O(\log(n))$
- 删除: $O(\log(n))$
- 第 k 小: $O(1)$ 前提: 每次操作 k 变化不大

空间复杂度: $O(n)$

用途: 双指针中位数

模板题: [2023ICPC-Jinan-Regional K. Rainbow Subarray](#)

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template<typename T>
5 struct SmartSet {
6     std::multiset<T> small, large;
7     T smallSum, largeSum;
8     SmartSet(): small(), large(), smallSum(), largeSum() {}
9     void insert(T val) {
10         if(small.empty() || val > *small.rbegin()) {

```

```

11             large.insert(val);
12             largeSum += val;
13         } else {
14             small.insert(val);
15             smallSum += val;
16         }
17     }
18     void erase(T val) {
19         if(small.contains(val)) {
20             smallSum -= val;
21             small.extract(val);
22         } else if(large.contains(val)) {
23             largeSum -= val;
24             large.extract(val);
25         }
26     }
27     void balance(int k) {
28         k = std::max(0, std::min(k, size()));
29         while(small.size() > k) {
30             T val = *small.rbegin();
31             smallSum -= val;
32             largeSum += val;
33             large.insert(val);
34             small.extract(val);
35         }
36         while(small.size() < k) {
37             T val = *large.begin();
38             smallSum += val;
39             largeSum -= val;
40             small.insert(val);
41             large.extract(val);
42         }
43     }
44     int size() {
45         return small.size() + large.size();
46     }
47     int smallSize(int k) {
48         balance(k);
49         return small.size();
50     }
51     int largeSize(int k) {
52         balance(k);
53         return large.size();
54     }
55     T kth(int k) {
56         balance(k);
57         return *small.rbegin();
58     }

```

```

59 T getSmallSum(int k) {
60     balance(k);
61     return smallSum;
62 }
63 T getLargeSum(int k) {
64     balance(k);
65     return largeSum;
66 }
67 };
68
69 void solve() {
70     i64 n, k;
71     std::cin >> n >> k;
72     std::vector<i64> v(n + 1);
73     for(int i = 1; i <= n; ++i) {
74         std::cin >> v[i];
75         v[i] = v[i] - i + n;
76     }
77     SmartSet<i64> sst;
78     int ans = 1;
79     for(int i = 1, j = 1; j <= n; ++j) {
80         sst.insert(v[j]);
81         while(true) {
82             int len = (j - i + 1);
83             int mid = (len + 1) / 2;
84             i64 target = sst.kth(mid);
85             i64 res = 1LL * target * (sst.smallSize(mid) - sst.largeSize(mid)) + sst
.getLargeSum(mid) - sst.getSmallSum(mid);
86             if(res > k) {
87                 sst.erase(v[i]);
88                 ++i;
89             } else {
90                 ans = std::max(ans, j - i + 1);
91                 break;
92             }
93         }
94     }
95 }
96 std::cout << ans << '\n';
97 }
98
99
100 int main() {
101     std::ios::sync_with_stdio(false);
102     std::cin.tie(nullptr);
103     int T = 1;
104     std::cin >> T;
105     while(T--) {

```

```

106         solve();
107     }
108 }

```

3.9 笛卡尔树

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template<typename T, typename _Compare = std::less<T>>
5 struct Cartesian {
6     Cartesian() = default;
7     Cartesian(const std::vector<T> &v, _Compare cmp = _Compare())
8         : ls(v.size(), -1), rs(v.size(), -1), cmp(std::move(cmp)) {
9         std::stack<int> stk;
10         for(int i = 0; i < v.size(); ++i) {
11             while(!stk.empty() && cmp(v[stk.top()], v[i])) {
12                 stk.pop();
13             }
14             if(stk.empty()) {
15                 ls[i] = root;
16                 root = i;
17             } else {
18                 ls[i] = rs[stk.top()];
19                 rs[stk.top()] = i;
20             }
21             stk.push(i);
22         }
23     }
24     _Compare cmp;
25     int root = -1;
26     std::vector<int> ls, rs;
27 };
28
29 int main() {
30     std::ios::sync_with_stdio(false);
31     std::cin.tie(nullptr);
32     int n;
33     std::cin >> n;
34     std::vector<int> v(n);
35     for(int i = 0; i < n; ++i) {
36         std::cin >> v[i];
37     }
38     Cartesian<int, std::greater<>> tree(v);
39     i64 lans = 0, rans = 0;
40     for(int i = 0; i < n; ++i) {
41         lans ^= 1LL * (i + 1) * (tree.ls[i] + 2);
42     }

```

```

43     for(int i = 0; i < n; ++i) {
44         rans ^= 1LL * (i + 1) * (tree.rs[i] + 2);
45     }
46     std::cout << lans << ' ' << rans << '\n';
47     return 0;
48 }

```

3.10 波纹疾走树

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3 using u32 = uint32_t;
4 using u64 = uint64_t;
5
6 struct BitRank {
7     std::vector<u64> block; // 管理一行一行的bit
8     std::vector<u32> count;
9     BitRank() {}
10
11     void resize(u32 num) { // 位向量长度
12         block.resize((num + 1) >> 6 + 1, 0);
13         count.resize(block.size(), 0);
14     }
15     void set(u32 i, u64 val) { // 设置i位bit
16         block[i >> 6] |= (val << (i & 63));
17     }
18     void build() {
19         for (u32 i = 1; i < block.size(); i++) {
20             count[i] = count[i - 1] + std::popcount(block[i - 1]);
21         }
22     }
23     u32 rank1(u32 i) { // [0, i) 1的个数
24         return count[i >> 6] + std::popcount(block[i >> 6] & ((1ULL << (i & 63)) - 1ULL));
25     }
26     u32 rank1(u32 i, u32 j) { // [i, j) 1的个数
27         return rank1(j) - rank1(i);
28     }
29     u32 rank0(u32 i) { // [0, i) 0的个数
30         return i - rank1(i);
31     }
32     u32 rank0(u32 i, u32 j) { // [i, j) 0的个数
33         return rank0(j) - rank0(i);
34     }
35 };
36
37 template<typename T> //要求非负整数
38 struct WaveletMatrix {

```

```

39     WaveletMatrix() = default;
40     WaveletMatrix(std::vector<T> vec) : WaveletMatrix(vec, std::ranges::max(vec) + 1) {}
41     // sigma: 字母表大小(字符串的话), 数字序列的话是数的种类
42     WaveletMatrix(std::vector<T> vec, T sigma) {
43         height = (sigma == 1) ? 1 : (64 - std::countl_zero<u64>(sigma - 1));
44         B.resize(height), pos.resize(height);
45         for (u32 i = 0; i < height; ++i) {
46             B[i].resize(vec.size());
47             for (u32 j = 0; j < vec.size(); ++j) {
48                 B[i].set(j, get(vec[j], height - i - 1));
49             }
50             B[i].build();
51             auto it = stable_partition(vec.begin(), vec.end(), [&](T c) {
52                 return !get(c, height - i - 1);
53             });
54             pos[i] = it - vec.begin();
55         }
56     }
57     T get(T val, int i) {
58         return (val >> i) & 1;
59     }
60     int rank(int l, int r, T val) { // [l, r] 中val出现的频率
61         return rank(r, val) - rank(l - 1, val);
62     }
63     int rank(int i, T val) { // [0, i] 中val出现的频率
64         ++i;
65         int p = 0;
66         for (u32 j = 0; j < height; ++j) {
67             if (get(val, height - j - 1)) {
68                 p = pos[j] + B[j].rank1(p);
69                 i = pos[j] + B[j].rank1(i);
70             } else {
71                 p = B[j].rank0(p);
72                 i = B[j].rank0(i);
73             }
74         }
75         return i - p;
76     }
77     T kth(int l, int r, int k) { // [l, r] 中k小
78         ++r;
79         T res = 0;
80         for (u32 i = 0; i < height; ++i) {
81             int j = B[i].rank0(l, r);
82             if (j >= k) {
83                 l = B[i].rank0(l);
84                 r = B[i].rank0(r);
85             } else {

```

```

86         l = pos[i] + B[i].rank1(l);
87         r = pos[i] + B[i].rank1(r);
88         k -= j;
89         res |= (T(1) << (height - i - 1));
90     }
91 }
92 return res;
93 }
94 int rangeFreq(int l, int r, T a, T b) { // [l,r] 在[a, b] 值域的数字个数
95     return rangeFreq(l, r + 1, a, b + 1, 0, T(1) << height, 0);
96 }
97 int rangeFreq(int i, int j, T a, T b, T l, T r, int x) {
98     if (i == j || r <= a || b <= l) return 0;
99     T mid = (l + r) >> 1;
100    if (a <= l && r <= b) {
101        return j - i;
102    } else {
103        int left = rangeFreq(B[x].rank0(i), B[x].rank0(j), a, b, l, mid, x + 1);
104        int right = rangeFreq(pos[x] + B[x].rank1(i), pos[x] + B[x].rank1(j), a,
105        b, mid, r, x + 1);
106        return left + right;
107    }
108 }
109 // [l,r] 在[a,b] 值域内存在的最小值是什么, 不存在返回-1
110 T rangeMin(int l, int r, T a, T b) {
111     return rangeMin(l, r + 1, a, b + 1, 0, T(1) << height, 0, 0);
112 }
113 T rangeMin(int i, int j, T a, T b, T l, T r, int x, T val) {
114     if (i == j || r <= a || b <= l) return -1;
115     if (r - l == 1) return val;
116     T mid = (l + r) >> 1;
117     T res = rangeMin(B[x].rank0(i), B[x].rank0(j), a, b, l, mid, x + 1, val);
118     if (res >= 0) return res;
119     return rangeMin(pos[x] + B[x].rank1(i), pos[x] + B[x].rank1(j), a, b, mid, r
120     , x + 1, val + (T(1) << (height - x - 1)));
121 }
122 T rangeMax(int l, int r, T a, T b) {
123     return rangeMax(l, r + 1, a, b + 1, 0, T(1) << height, 0, 0);
124 }
125 T rangeMax(int i, int j, T a, T b, T l, T r, int x, T val) {
126     if (i == j || r <= a || b <= l) return -1;
127     if (r - l == 1) return val;
128     T mid = (l + r) >> 1;
129     T res = rangeMax(pos[x] + B[x].rank1(i), pos[x] + B[x].rank1(j), a, b, mid,
130     r, x + 1, val + (T(1) << (height - x - 1)));
131     if (res >= 0) return res;
132     return rangeMax(B[x].rank0(i), B[x].rank0(j), a, b, l, mid, x + 1, val);
133 }

```

```

131     u32 height;
132     std::vector<BitRank> B;
133     std::vector<int> pos;
134 };
135
136 // 波状疾走树(区间第k小, 区间val出现的频率, 区间在值域出现的次数和最小值)
137 // https://www.luogu.com.cn/problem/P3834
138 int main() {
139     std::ios::sync_with_stdio(false);
140     std::cin.tie(0);
141     int n, q;
142     std::cin >> n >> q;
143     std::vector<i64> v(n + 1);
144     for(int i = 1; i <= n; ++i) {
145         std::cin >> v[i];
146     }
147     WaveletMatrix<i64> wlm(v);
148     for(int i = 1; i <= q; ++i) {
149         int l, r, k;
150         std::cin >> l >> r >> k;
151         std::cout << wlm.kth(l, r, k) << '\n';
152     }
153     return 0;
154 }

```

3.11 欧拉序

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long i64;
4
5 struct SparseTable {
6     SparseTable() = default;
7     vector<int> v;
8     vector<vector<int>> st;
9     void init(vector<int> &v_) {
10         v = v_;
11         int k = __lg(v.size());
12         st = vector<vector<int>>(k + 1, vector<int>(v.size()));
13         iota(st[0].begin(), st[0].end(), 0);
14         for (int i = 0; i < k; ++i) {
15             for (int j = 0; j + (1 << (i + 1)) - 1 < v.size(); ++j) {
16                 if (v[st[i][j]] < v[st[i][j + (1 << i)]]
17                     st[i + 1][j] = st[i][j];
18                 else
19                     st[i + 1][j] = st[i][j + (1 << i)];
20             }
21         }

```

```

22     }
23     int query_id(int l, int r) {
24         int t = __lg(r - l + 1);
25         if (v[st[t][l]] < v[st[t][r + 1 - (1 << t)]])
26             return st[t][l];
27         else
28             return st[t][r + 1 - (1 << t)];
29     }
30 };
31
32 struct Euler_tours {
33     int n, cnt = 0;
34     vector<vector<int>> graph;
35     vector<int> et_dep, id, et;
36     SparseTable st; //节点个数, 图
37     Euler_tours(int n) : n(n), graph(n + 1), id(n + 1), et_dep(2 * n), et(2 * n) {}
38
39     void add_edg(int u, int v) {
40         graph[u].push_back(v);
41         graph[v].push_back(u);
42     }
43
44     void dfs(int u, int fa, int dep) {
45         et_dep[++cnt] = dep;
46         et[cnt] = u;
47         id[u] = cnt;
48         for (auto v : graph[u]) {
49             if (v != fa) {
50                 dfs(v, u, dep + 1);
51                 et_dep[++cnt] = dep;
52                 et[cnt] = u;
53             }
54         }
55         return;
56     }
57
58     void init(int root = 1) {
59         dfs(root, 0, 1);
60         st.init(et_dep);
61     }
62
63     int lca(int u, int v) {
64         int idu = id[u];
65         int idv = id[v];
66         if (idu > idv) {
67             swap(idu, idv);
68         }
69         int idlca = st.query_id(idu, idv);

```

```

70         return et[idlca];
71     }
72 };
73
74 void solve() {
75     int n, q, root;
76     cin >> n >> q >> root;
77     Euler_tours et(n);
78     for (i64 i = 1; i < n; i++) {
79         i64 u, v;
80         cin >> u >> v;
81         et.add_edg(u, v);
82     }
83     et.init(root);
84     while (q--) {
85         i64 u, v;
86         cin >> u >> v;
87         cout << et.lca(u, v) << "\n";
88     }
89     return;
90 }
91
92 int main() {
93     ios::sync_with_stdio(false);
94     cin.tie(0), cout.tie(0);
95     i64 T = 1;
96     // cin >> T;
97     while (T--) {
98         solve();
99     }
100     return 0;

```

4 字符串

4.1 字符串哈希

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 struct H64 {
5     H64() = default;
6     H64(i64 v) : v(v) {}
7     H64 operator+(const H64 &o) const {
8         return (v + o.v >= MOD ? v + o.v - MOD : v + o.v);
9     }
10    H64 operator-(const H64 &o) const {
11        return (o.v > v ? v + MOD - o.v : v - o.v);

```

```

12 }
13 H64 operator*(const H64 &o) const {
14     __int128 r = (__int128)v * o.v;
15     return H64((r >> 61) + (r & MOD));
16 }
17 auto operator<=>(const H64& o) const = default;
18 i64 v = 0;
19 constexpr static i64 MOD = (1LL << 61) - 1;
20 };
21
22 struct Hash {
23     Hash() = default;
24     Hash(const Hash &hs) : h(hs.h), n(1) {}
25     Hash(const int &val) : h(H64(val)), n(1) {}
26     Hash(const std::string &s) : n(s.size()) {
27         for(int i = 0; i < (int)s.size(); ++i) {
28             h = h + H64(s[i]) * fac(s.size() - i - 1);
29         }
30     }
31     Hash(const H64 &h, const int &n) : h(h), n(n) {}
32     auto operator<=>(const Hash& o) const = default;
33     Hash operator+(const Hash &o) {
34         return {h * fac(o.n) + o.h, n + o.n};
35     }
36     Hash operator-(const Hash &o) {
37         return {h - o.h * fac(n - o.n), n - o.n};
38     }
39     Hash operator+=(const Hash &o) {
40         return *this = (*this + o);
41     }
42     Hash operator-=(const Hash &o) {
43         return *this = (*this - o);
44     }
45     static H64 fac(int x) {
46         if(x >= f.size()) {
47             int l = f.size();
48             f.resize(x + 1);
49             for(int i = l; i < f.size(); ++i) {
50                 f[i] = f[i - 1] * BASE;
51             }
52         }
53         return f[x];
54     }
55     int n{};
56     H64 h{};
57     constexpr static i64 BASE = 114514;
58     inline static std::vector<H64> f{H64(1)};
59 };

```

```

60
61 int main() {
62     std::ios::sync_with_stdio(false);
63     std::cin.tie(nullptr);
64     int n;
65     std::cin >> n;
66     std::set<Hash> st;
67     for(int i = 0; i < n; ++i) {
68         std::string s;
69         std::cin >> s;
70         st.insert(Hash(s));
71     }
72     std::cout << st.size() << '\n';
73     return 0;
74 }

```

4.2 字典树

```

1 #include <bits/stdc++.h>
2
3 template<int N = 26>
4 struct Trie {
5     Trie() = default;
6     Trie(const std::string &rgx) : tree(1) {
7         int k = 0;
8         for(int i = 0, dx; i < rgx.size(); i += dx + 1) {
9             dx = 2 * (i + 2 < rgx.size() && rgx[i + 1] == '-');
10            for(int j = rgx[i]; j <= rgx[i + dx]; ++j) {
11                dict[j] = k++;
12            }
13        }
14    }
15    void insert(const std::string &s) {
16        int cur = 0;
17        for(const auto &ch : s) {
18            int &nxt = tree[cur][dict[ch]];
19            if(nxt == 0) nxt = ++idx;
20            cur = nxt;
21            if(tree.size() <= cur) {
22                tree.resize(cur + s.size());
23                tot.resize(cur + s.size());
24            }
25            tot[cur]++;
26        }
27    }
28    int find(const std::string &s) {
29        int cur = 0;
30        for(const auto &ch : s) {

```

```

31         int &nxt = tree[cur][dict[ch]];
32         if(nxt == 0) return 0;
33         cur = nxt;
34     }
35     return tot[cur];
36 }
37
38 int idx = 0;
39 std::vector<int> tot;
40 std::array<int, 128> dict{};
41 std::vector<std::array<int, N>> tree;
42 };
43
44 void solve() {
45     int n, q;
46     std::cin >> n >> q;
47     Trie<62> t("0-9A-Za-z");
48     for(int i = 1; i <= n; ++i) {
49         std::string s;
50         std::cin >> s;
51         t.insert(s);
52     }
53     for(int i = 1; i <= q; ++i) {
54         std::string s;
55         std::cin >> s;
56         std::cout << t.find(s) << '\n';
57     }
58 }
59
60 int main() {
61     std::ios::sync_with_stdio(false);
62     std::cin.tie(nullptr);
63     int T = 1;
64     std::cin >> T;
65     while(T--) {
66         solve();
67     }
68     return 0;
69 }

```

4.3 KMP

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 std::vector<int> kmp(const std::string &s) {
5     std::vector<int> v(s.size());
6     for(int i = 1, j = 0; i < s.size(); ++i) {

```

```

7         while(j > 0 && s[i] != s[j]) {
8             j = v[j - 1];
9         }
10        if(s[i] == s[j]) {
11            ++j;
12        }
13        v[i] = j;
14    }
15    return v;
16 }
17
18 int main() {
19     std::ios::sync_with_stdio(false);
20     std::cin.tie(nullptr);
21     std::string s, p;
22     std::cin >> s >> p;
23     auto v = kmp(p + "#" + s);
24     for(int i = p.size() + 1; i < v.size(); ++i) {
25         if(v[i] == p.size()) {
26             std::cout << i - 2 * p.size() + 1 << '\n';
27         }
28     }
29     for(int i = 0; i < p.size(); ++i) {
30         std::cout << v[i] << " \n"[i + 1 == p.size()];
31     }
32     return 0;
33 }

```

4.4 EXKMP

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 int main() {
5     std::ios::sync_with_stdio(false);
6     std::cin.tie(nullptr);
7     std::string a, b;
8     std::cin >> a >> b;
9     int n = a.size(), m = b.size();
10    a = '#' + a, b = '#' + b;
11    std::vector<int> z(m + 1), p(n + 1);
12    z[1] = m;
13    for(int i = 2, l = 0, r = 0; i <= m; ++i) {
14        if(i <= r) {
15            z[i] = std::min(z[i - l + 1], r - i + 1);
16        }
17        while(i + z[i] <= m && b[i + z[i]] == b[1 + z[i]]) {
18            z[i]++;

```

```

19     }
20     if(i + z[i] - 1 > r) {
21         l = i, r = i + z[i] - 1;
22     }
23 }
24 for(int i = 1, l = 0, r = 0; i <= n; ++i) {
25     if(i <= r) {
26         p[i] = std::min(z[i - l + 1], r - i + 1);
27     }
28     while(1 + p[i] <= m && i + p[i] <= n && b[1 + p[i]] == a[i + p[i]]) {
29         p[i]++;
30     }
31     if(i + p[i] - 1 > r) {
32         l = i, r = i + p[i] - 1;
33     }
34 }
35 i64 ans1 = 0, ans2 = 0;
36 for(int i = 1; i <= m; ++i) {
37     ans1 ^= 1LL * i * (z[i] + 1);
38 }
39 for(int i = 1; i <= n; ++i) {
40     ans2 ^= 1LL * i * (p[i] + 1);
41 }
42 std::cout << ans1 << '\n' << ans2 << '\n';
43 return 0;
44 }

```

```

19     }
20     while(t[i - v[i] - 1] == t[i + v[i] + 1] && 0 <= i - v[i] - 1 && i + v[i] +
21         1 < n) {
22         ++v[i];
23     }
24     if(i + v[i] > mid + v[mid]) {
25         mid = i;
26     }
27     return v;
28 }
29
30 int main() {
31     std::ios::sync_with_stdio(false);
32     std::cin.tie(nullptr);
33     std::string s;
34     std::cin >> s;
35     std::vector<int> v = manacher(s);
36     int ans = 0;
37     for(int i = 0; i < v.size(); ++i) {
38         ans = std::max(ans, v[i]); //求最长回文子串
39         std::cout << v[i] << " \n"[i == v.size() - 1];
40     }
41     std::cout << ans << '\n';
42     return 0;
43 }

```

4.5 马拉车

```

1 #include <bits/stdc++.h>
2
3 //马拉车(manacher)
4 //https://www.luogu.com.cn/problem/P3805
5
6 // 以第i个数轴的最大回文 v[2 * i + 1]
7 // 以第i个数和i+1个数中间轴的最大回文 v[2 * i + 2]
8 // 以[L, R] 区间中轴的最大回文为v[L + R + 1]
9 std::vector<int> manacher(const std::string& s) {
10     int n = 2 * s.length() + 1;
11     std::string t(n, '#'); //处理字符串
12     for(int i = 0; i < s.length(); ++i) {
13         t[2 * i + 1] = s[i];
14     }
15     std::vector<int> v(n); //记录回文半径 [l, r] <=> [mid - v[mid], mid + v[mid]]
16     for(int i = 0, mid = 0; i < n; ++i) { // mid为回文中心
17         if(i <= mid + v[mid]) {
18             v[i] = std::min(v[2 * mid - i], mid + v[mid] - i); // (t + i) / 2 = mid
19             <=> t = 2 * mid - i;

```

5 计算几何

5.1 凸包

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 constexpr long double EPS = 1E-10;
5
6 template<typename T>
7 int sgn(T a){
8     return (a > EPS) - (a < -EPS);
9 }
10
11 template<typename T>
12 struct Point;
13
14 template<typename T>
15 struct Vector { //向量
16     T x = 0, y = 0;

```

```

17 Vector(const Point<T> &p) : x(p.x), y(p.y) {}
18 Vector(T x_ = 0, T y_ = 0) : x(x_), y(y_) {}
19 template<typename U> operator Vector<U>() {return Vector<U>(U(x), U(y));}
20 Vector operator+(const Vector<T> &o) const {return {x + o.x, y + o.y};}
21 Vector operator-(const Vector<T> &o) const {return {x - o.x, y - o.y};}
22 Vector operator*(T f) const {return {x * f, y * f};}
23 Vector operator/(T f) const {return {x / f, y / f};}
24 friend std::istream &operator>>(std::istream &is, Vector &p) {
25     return is >> p.x >> p.y;
26 }
27 friend std::ostream &operator<<(std::ostream &os, Vector p) {
28     return os << "(" << p.x << ", " << p.y << ")";
29 }
30 };
31
32 template<typename T>
33 struct Point { //点
34     T x = 0, y = 0;
35     Point(T x_ = 0, T y_ = 0) : x(x_), y(y_) {}
36     template<typename U> operator Point<U>() {return Point<U>(U(x), U(y));}
37     Point operator+(const Vector<T> &o) const {return {x + o.x, y + o.y};}
38     Vector<T> operator-(const Point<T> &o) const {return {x - o.x, y - o.y};}
39     Point operator-() const {return {-x, -y};}
40     Point operator*(T f) const {return {x * f, y * f};}
41     Point operator/(T f) const {return {x / f, y / f};}
42     friend Point operator*(T f, Point p) {return {p * f};}
43     bool operator==(const Point &o) const {
44         return sgn(x - o.x) == 0 && sgn(y - o.y) == 0;
45     }
46     constexpr std::strong_ordering operator<=>(const Point &o) const {
47         if(sgn(x - o.x) == 0) {
48             return sgn(y - o.y) <=> 0;
49         } else {
50             return sgn(x - o.x) <=> 0;
51         }
52     }
53     friend std::istream &operator>>(std::istream &is, Point &p) {
54         return is >> p.x >> p.y;
55     }
56     friend std::ostream &operator<<(std::ostream &os, Point p) {
57         return os << "(" << p.x << ", " << p.y << ")";
58     }
59 };
60
61 template<typename T>
62 struct Line { //直线
63     Point<T> s, t;
64     Line() = default;

```

```

65     Line(Point<T> _s, Point<T> _t) : s(_s), t(_t) {}
66 };
67
68 template<typename T>
69 struct Seg { //线段
70     Point<T> s, t;
71     Seg() = default;
72     Seg(Point<T> _s, Point<T> _t) : s(_s), t(_t) {}
73 };
74
75 template<typename T, typename U>
76 struct Circle { //圆
77     Point<T> o;
78     U r{};
79 };
80
81 template<typename T>
82 T dot(const Vector<T> &a, const Vector<T> &b) { //向量a和向量b的点积
83     return a.x * b.x + a.y * b.y;
84 }
85
86 template<typename T>
87 T dot(const Point<T> &a, const Point<T> &b, const Point<T> &c) { //向量a->b和向量a->
88     //c的点积
89     return dot(b - a, c - a);
90 }
91
92 template<typename T>
93 T cross(const Vector<T> &a, const Vector<T> &b) { //向量a和向量b的叉积
94     return a.x * b.y - a.y * b.x;
95 }
96
97 template<typename T>
98 T cross(const Point<T> &a, const Point<T> &b, const Point<T> &c) { //向量a->b和向量a
99     //->c的叉积
100     return cross(b - a, c - a);
101 }
102
103 template<typename T>
104 T len2(const Vector<T> &a) { //向量a的模长的平方
105     return a.x * a.x + a.y * a.y;
106 }
107
108 template<typename T>
109 long double len(const Vector<T> &a) { //向量a的模长
110     return sqrtl(len2(a));
111 }

```

```

111 template<typename T>
112 Vector<T> standardize(const Vector<T> &a) { //向量a的单位向量
113     return a / len(a);
114 }
115
116 template<typename T>
117 long double angle(Vector<T> a, Vector<T> b) { //求两向量夹角[0, pi]
118     return fabs(atan2l(cross(a, b), dot(a, b)));
119 }
120
121 template<typename T>
122 T dis2(const Point<T> &a, const Point<T> &b) { //点a和点b距离的平方（防精度损失）
123     return (b.x - a.x) * (b.x - a.x) + (b.y - a.y) * (b.y - a.y);
124 }
125
126 template<typename T>
127 long double dis(const Point<T> &a, const Point<T> &b) { //点a到点b距离
128     return sqrtl(dis2(a, b));
129 }
130
131 template<typename T>
132 Vector<T> rotate(const Vector<T> &a) { //向量a逆时针旋转pi/2
133     return {-a.y, a.x};
134 }
135
136 template<typename T>
137 Vector<T> rotate(const Vector<T> &a, long double rad) { //向量a逆时针旋转rad弧度
138     return {a.x * cosl(rad) - a.y * sinl(rad), a.x * sinl(rad) + a.y * cosl(rad)};
139 }
140
141 template<typename T>
142 Point<T> rotate(const Point<T> &a, const Point<T> &b, long double rad) { //点b绕点a
    逆时针旋转rad弧度
143     return {
144         (b.x - a.x) * cosl(rad) - (b.y - a.y) * sinl(rad) + a.x,
145         (b.x - a.x) * sinl(rad) + (b.y - a.y) * cosl(rad) + a.y
146     };
147 }
148
149 template<typename T>
150 bool intersect(const Seg<T> &a, const Seg<T> &b) { //线段a和线段b不严格相交，可以包
    含端点相交
151     return sgn(cross(a.s, a.t, b.s) * cross(a.s, a.t, b.t)) <= 0
152         && sgn(cross(b.s, b.t, a.s) * cross(b.s, b.t, a.t)) <= 0;
153 }
154
155 template<typename T>
156 bool intersectStrictly(const Seg<T> &a, const Seg<T> &b) { //线段a和线段b严格相交，

```

```

    不包含端点相交
157     return sgn(cross(a.s, a.t, b.s) * cross(a.s, a.t, b.t)) < 0
158         && sgn(cross(b.s, b.t, a.s) * cross(b.s, b.t, a.t)) < 0;
159 }
160
161 template<typename T>
162 Point<T> getNode(const Line<T> &a, const Line<T> &b) { //求直线a与直线b的交点（需要
    先判断是否相交）
163     T dx = cross(a.s, b.s, b.t) / cross(a.t - a.s, b.t - b.s);
164     return a.s + (a.t - a.s) * dx;
165 }
166
167 template<typename T>
168 Point<T> getNode(const Seg<T> &a, const Seg<T> &b) { //求线段a与线段b的交点（需要先
    判断是否相交）
169     T dx = cross(a.s, b.s, b.t) / cross(a.t - a.s, b.t - b.s);
170     return a.s + (a.t - a.s) * dx;
171 }
172
173 template<typename T>
174 Line<T> midseg(const Seg<T> &seg) { //求线段的垂直平分线
175     Point mid = (seg.s + seg.t) / 2;
176     return {mid, mid + rotate(seg.t - seg.s)};
177 }
178
179 template<typename T>
180 Circle<T, long double> getCircle(const Point<T> &p1, const Point<T> &p2, const Point
    <T> &p3) { //求三点所在圆
181     Line<T> l1 = midseg(Seg<T>{p1, p2});
182     Line<T> l2 = midseg(Seg<T>{p1, p3});
183     Point<T> o = getNode(l1, l2);
184     return {o, dis(o, p1)};
185 }
186
187 template<typename T>
188 Circle<T, long double> getCircle(const Point<T> &p1, const Point<T> &p2) { //求以两
    点为直径所在圆
189     Point<T> o = (p1 + p2) / 2;
190     return {o, dis(o, p1)};
191 }
192
193 template<typename T>
194 bool onSeg(const Point<T> &p, const Seg<T> &s) { //判断点是否在线段上
195     if(sgn(cross(s.s, s.t, p)) != 0) return false;
196     return std::min(s.s, s.t) <= p && p <= std::max(s.s, s.t);
197 }
198
199 template<typename T, typename U>

```

```

200 int pointOnCircle(const Point<T> &p, const Circle<T, U> &c) { //点和圆的位置关系, -1
    点在圆外, 0点在圆上, 1点在圆外,
    return sgn(c.r * c.r - dis2(p, c.o));
201 }
202 }
203
204 template<typename T>
205 std::vector<Point<T>> andrew(std::vector<Point<T>> v) { //Andrew求凸包
206     std::sort(v.begin(), v.end());
207     std::vector<Point<T>> stk;
208     for(int i = 0; i < v.size(); ++i) {
209         while(stk.size() > 1 && sgn(cross(stk[stk.size() - 2], stk.back(), v[i])) <=
            0) {
210             stk.pop_back();
211         }
212         stk.push_back(v[i]);
213     }
214     int t = stk.size();
215     for(int i = (int)v.size() - 2; i >= 0; --i) {
216         while(stk.size() > t && sgn(cross(stk[stk.size() - 2], stk.back(), v[i])) <=
            0) {
217             stk.pop_back();
218         }
219         stk.push_back(v[i]);
220     }
221     stk.pop_back();
222     return stk;
223 };
224
225 template<typename T>
226 std::vector<Point<T>> graham(std::vector<Point<T>> v) { //Graham求凸包
227     Point<T> base = *min_element(v.begin(), v.end());
228     std::ranges::sort(v, [&](auto p1, auto p2) ->bool {
229         if(sgn(cross(base, p1, p2)) == 0) return p1 < p2;
230         return sgn(cross(base, p1, p2)) == -1;
231     });
232     std::vector<Point<T>> stk;
233     for(int i = 0; i < v.size(); ++i) {
234         while(stk.size() > 1 && sgn(cross(stk[stk.size() - 2], stk.back(), v[i])) >=
            0) {
235             stk.pop_back();
236         }
237         stk.push_back(v[i]);
238     }
239     return stk;
240 }
241
242 template<typename T>
243 std::vector<Line<T>> halfcut(std::vector<Line<T>> lines) { //半平面交, 默认左侧

```

```

244     std::sort(lines.begin(), lines.end(), [&](const auto &l1, const auto &l2) ->bool
        {
245         auto v1 = l1.t - l1.s, v2 = l2.t - l2.s;
246         auto a1 = atan2(v1.y, v1.x), a2 = atan2(v2.y, v2.x);
247         if(sgn(a1 - a2) != 0) return sgn(a1 - a2) == -1;
248         return sgn(cross(l1.s, l1.t, l2.t)) < 0;
249     });
250     int l = 0, r = -1;
251     std::vector<Line<T>> ls(lines.size());
252     for(const auto &line : lines) {
253         if(r >= l && sgn(cross(line.t - line.s, ls[r].t - ls[r].s)) == 0) continue;
254         while(r > l && sgn(cross(line.s, line.t, getNode(ls[r], ls[r - 1]))) == -1)
            r--;
255         while(r > l && sgn(cross(line.s, line.t, getNode(ls[l], ls[l + 1]))) == -1)
            l++;
256         ls[++r] = line;
257     }
258     while(r > l + 1 && sgn(cross(ls[l].s, ls[l].t, getNode(ls[r], ls[r - 1]))) ==
        -1) r--;
259     return std::vector<Line<T>>(ls.begin() + l, ls.begin() + r + 1);
260 }
261
262 template<typename T>
263 std::vector<Point<T>> linesToPoints(const std::vector<Line<T>> &lines) { //直线式凸
    包转为点凸包
264     std::vector<Point<T>> v;
265     for(int i = 0; i < lines.size(); ++i) {
266         v.push_back(getNode(lines[i], lines[(i + 1) % lines.size()]));
267     }
268     return v;
269 };
270
271 template<typename T>
272 long double area(const std::vector<Point<T>> &v) { //求凸包面积
273     long double ans = 0;
274     for(int i = 1; i + 1 < v.size(); ++i) {
275         ans += cross(v[0], v[i], v[i + 1]);
276     }
277     ans /= 2;
278     return ans;
279 }
280
281 template<typename T>
282 T diameter2(const std::vector<Point<T>> &v) { //旋转卡壳求凸包直径的平方
283     int n = v.size();
284     T res = 0;
285     for(int i = 0, j = 1; i < n; ++i) {
286         while(sgn(cross(v[i], v[(i + 1) % n], v[j]) - cross(v[i], v[(i + 1) % n], v

```

```

    [(j + 1) % n])) <= 0) {
287         j = (j + 1) % n;
288     }
289     res = std::max({res, dis2(v[i], v[j]), dis2(v[(i + 1) % n], v[j])});
290 }
291 return res;
292 }
293
294 template<typename T>
295 long double diameter(const std::vector<Point<T>> &v) { //旋转卡壳求凸包直径
296     return sqrtl(diameter2(v));
297 }
298
299 template<typename T>
300 long double mindistance(std::vector<Point<T>> v) { //平面最近点对O(nlog(n))
301     std::sort(v.begin(), v.end());
302     long double d = 1E18;
303     auto cmp = [](const Point<T>& a, const Point<T>& b) {
304         return a.y < b.y;
305     };
306     std::multiset<Point<T>, decltype(cmp)> st;
307     for(int i = 0, j = 0; i < v.size(); ++i) {
308         while(j < i && sgn(v[j].x - v[i].x + d) <= 0) {
309             st.erase(v[j]);
310             ++j;
311         }
312         for(auto it = st.lower_bound({v[i].x, v[i].y - T(d)}); it != st.end() && sgn
(it->y - v[i].y - T(d + 1)) <= 0; ++it) {
313             d = std::min(d, dis(v[i], *it));
314         }
315         st.insert(v[i]);
316     }
317     return d;
318 }
319
320 template<typename T>
321 long double grith(const std::vector<Point<T>> &v) { //求凸包周长
322     long double ans = 0;
323     for(int i = 0; i < v.size(); ++i) {
324         ans += dis(v[i], v[(i + 1) % v.size()]);
325     }
326     return ans;
327 }
328
329 void solve() {
330 }
331 }
332

```

```

333 int main() {
334     std::ios::sync_with_stdio(false);
335     std::cin.tie(nullptr);
336     int T = 1;
337     // std::cin >> T;
338     while(T--) {
339         solve();
340     }
341     return 0;
342 }

```

6 杂项

6.1 取整

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 template<typename T>
5 T floorDiv(T a, T b) {
6     return a / b - (((a < 0) ^ (b < 0)) && a % b != 0);
7 }
8
9 template<typename T>
10 T ceilDiv(T a, T b) {
11     return a / b + (!((a < 0) ^ (b < 0)) && a % b != 0);
12 }
13
14 template<typename T>
15 T roundDiv(T a, T b) {
16     return a / b + (abs(a % b) * 2 >= abs(b) ? ((a < 0) ^ (b < 0) ? -1 : 1) : 0);
17 }
18
19 int main() {
20     std::ios::sync_with_stdio(false);
21     std::cin.tie(nullptr);
22     return 0;
23 }

```

6.2 日期公式

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 constexpr int BASE = 1900;
5 constexpr int DAY[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

```

```

6 constexpr int PRE[] = {0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334, 365};
7
8 bool isLeapYear(int y) {
9     return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
10 }
11
12 int dayId(int y, int m, int d) { // (y, m, d) 离 (BASE, 1, 1) 的天数, 从0开始
13     int res = (y - BASE) * 365 + PRE[m - 1] + d - 1;
14     int y1 = BASE - 1, y2 = y - 1;
15     res += (y2 / 4 - y2 / 100 + y2 / 400);
16     res -= (y1 / 4 - y1 / 100 + y1 / 400);
17     return res + (m > 2 && isLeapYear(y));
18 }
19
20 int wday(int y, int m, int d) { //求周几
21     return (d + 2 * m + 3 * (m + 1) / 5 + y + y / 4 - y / 100 + y / 400) % 7 + 1;
22 }
23
24 std::array<int, 3> idToDate(i64 id) { // 通过 dayId 反向获取日期
25     int y = BASE + (id * 10000) / 3652425;
26     while (dayId(y, 1, 1) > id) y--;
27     while (dayId(y + 1, 1, 1) <= id) y++;
28     int m = (id - dayId(y, 1, 1)) * 1000 / 30419 + 1;
29     while (dayId(y, m, 1) > id) m--;
30     while (dayId(y, m + 1, 1) <= id) m++;
31     int d = id - dayId(y, m, 1) + 1;
32     return {y, m, d};
33 }
34
35 int main() {
36     std::ios::sync_with_stdio(false);
37     std::cin.tie(nullptr);
38
39     return 0;
40 }

```

6.3 动态 bitset

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 struct dynamic_bitset {
5     dynamic_bitset() = default;
6     dynamic_bitset(const uint64_t &n) : v(1, n), _count(std::popcount(n)), _size(64)
7     {}
8     dynamic_bitset(const int sz, bool bit) {
9         resize(sz);
10         if(bit) {

```

```

11             set();
12         }
13     }
14     dynamic_bitset(const std::string &s) {
15         resize(s.size());
16         for(int i = 0; i < s.size(); ++i) {
17             if(s[i] == '0') continue;
18             set(i);
19         }
20     }
21 #define blk(x) (x >> 6)
22 #define idx(x) (x & 63)
23     bool operator==(const dynamic_bitset &o) const {
24         return _size == o._size && v == o.v;
25     }
26     bool operator!=(const dynamic_bitset &o) const {
27         return !(*this == o);
28     }
29     dynamic_bitset operator&=(const dynamic_bitset &o) {
30         return combine(o, std::bit_and());
31     }
32     dynamic_bitset operator|=(const dynamic_bitset &o) {
33         return combine(o, std::bit_or());
34     }
35     dynamic_bitset operator^=(const dynamic_bitset &o) {
36         return combine(o, std::bit_xor());
37     }
38     friend dynamic_bitset operator&(dynamic_bitset x, const dynamic_bitset &y) {
39         return x &= y;
40     }
41     friend dynamic_bitset operator|(dynamic_bitset x, const dynamic_bitset &y) {
42         return x |= y;
43     }
44     friend dynamic_bitset operator^(dynamic_bitset x, const dynamic_bitset &y) {
45         return x ^= y;
46     }
47
48     dynamic_bitset operator~() const {
49         dynamic_bitset res;
50         res.resize(_size);
51         for (size_t i = 0; i < v.size(); ++i) {
52             res.v[i] = ~v[i];
53         }
54         if(!v.empty()) {
55             res.v.back() &= get(0, idx(_size - 1));
56         }
57         res._count = _size - _count;

```

```

58     return res;
59 }
60 dynamic_bitset operator>>(int dx) const {
61     dynamic_bitset res;
62     res.resize(std::max(0, (int)_size - dx));
63     for(int i = 0; i < res.v.size(); ++i) {
64         int l = (i << 6) + dx;
65         int r = std::min((int)_size - 1, l + 63);
66         assert(blk(l) < v.size());
67         res.v[i] = (v[blk(l)] >> idx(l)) | (v[blk(r)] << idx(r));
68         res._count += std::popcount(res.v[i]);
69     }
70     res.reset(res._size, (res.v.size() << 6) - 1);
71     return res;
72 }
73 dynamic_bitset operator<<(int dx) const {
74     dynamic_bitset res;
75     res.resize(_size + dx);
76     for(int i = 0; i < v.size(); ++i) {
77         int l = (i << 6) + dx;
78         int r = std::min((int)res._size - 1, l + 63);
79         res.v[blk(l)] |= v[i] << idx(dx);
80         res.v[blk(r)] |= v[i] >> (64 - idx(dx));
81     }
82     res._count = _count;
83     return res;
84 }
85
86 dynamic_bitset sub(int l, int r) const {
87     dynamic_bitset res = *this >> l;
88     res.resize(r - l + 1);
89     return res;
90 }
91 bool operator[](const int &pos) const {
92     return v[blk(pos)] >> idx(pos) & 1ULL;
93 }
94
95 unsigned long to_ulong() const { return v.empty() ? 0 : (unsigned long)v[0]; }
96 unsigned long long to_ullong() const { return v.empty() ? 0 : (unsigned long
long)v[0]; }
97 std::string to_string() const {
98     std::string res(_size, '0');
99     for(int i = 0; i < _size; ++i) {
100         res[i] = '0' + (*this)[i];
101     }
102     return res;
103 }
104 void set(int l, int r) {

```

```

105     update(l, r, [](auto &x, auto mask) { x |= mask; });
106 }
107 void reset(int l, int r) {
108     update(l, r, [](auto &x, auto mask) { x &= ~mask; });
109 }
110 void flip(int l, int r) {
111     update(l, r, [](auto &x, auto mask) { x ^= mask; });
112 }
113 void set() { set(0, _size - 1); }
114 void reset() { reset(0, _size - 1); }
115 void flip() { flip(0, _size - 1); }
116 void set(int pos) { set(pos, pos); }
117 void reset(int pos) { reset(pos, pos); }
118 void flip(int pos) { flip(pos, pos); }
119
120 void push_back(bool bit) {
121     resize(_size + 1);
122     if(!bit) return;
123     set(_size - 1);
124 }
125 void pop_back() {
126     resize(_size - 1);
127 }
128
129 size_t size() const { return _size; }
130 size_t count() const { return _count; }
131 size_t num_blocks() const { return v.size(); }
132 bool any() const { return _count != 0; }
133 bool all() const { return _count == _size; }
134 bool none() const { return _count == 0; }
135
136 void resize(size_t sz) {
137     if(sz < _size) {
138         reset(sz, _size - 1);
139     }
140     v.resize((sz + 63) / 64);
141     _size = sz;
142 }
143
144 uint64_t get(int l, int r) const {
145     return (~0ULL << l) & (~0ULL >> (63 - r));
146 }
147 void update(int l, int r, auto func) {
148     for(int i = blk(l); i <= blk(r); ++i) {
149         _count -= std::popcount(v[i]);
150         int bl = i == blk(l) ? idx(l) : 0;
151         int br = i == blk(r) ? idx(r) : 63;
152         func(v[i], get(bl, br));

```

```

153     _count += std::popcount(v[i]);
154 }
155 }
156 dynamic_bitset combine(const dynamic_bitset &o, auto func) {
157     resize(std::max(_size, o._size));
158     _count = 0;
159     for(int i = 0; i < v.size(); ++i) {
160         v[i] = func(v[i], i < o.v.size() ? o.v[i] : 0ULL);
161         _count += std::popcount(v[i]);
162     }
163     return *this;
164 }
165 friend std::ostream &operator<<(std::ostream &os, const dynamic_bitset &db){
166     for(int i = (int)db.v.size() - 1; i >= 0; --i) {
167         if(i + 1 == db.v.size()) {
168             for(int j = (int)db._size - 1; j >= i * 64; --j) {
169                 os << db[j];
170             }
171         } else {
172             os << std::bitset<64>(db.v[i]);
173         }
174     }
175     return os;
176 }
177 std::vector<uint64_t> v;
178 size_t _count = 0;
179 size_t _size = 0;
180
181 #undef blk
182 #undef idx
183 };
184
185 int main() {
186     dynamic_bitset A("10"), B("100");
187     return 0;
188 }

```

6.4 高精度

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 // using i128= __int128;
5 // std::istream&operator>>(std::istream &is,i128 &n){
6 //     std::string s;is>>s;
7 //     n=0;
8 //     for(char i:s) n=n*10+i-'0';
9 //     return is;

```

```

10 // }
11 // std::ostream &operator<<(std::ostream &os,i128 n){
12 //     std::string s;
13 //     while(n){
14 //         s+='0'+n%10;
15 //         n/=10;
16 //     }
17 //     std::reverse(s.begin(),s.end());
18 //     return os<<s;
19 // }
20
21 struct Bigint {
22     std::string a;
23     int sign;
24     Bigint() {}
25     Bigint(std::string b) {
26         (*this) = b;
27     }
28     int size() {
29         return a.size();
30     }
31     Bigint normalize(int newSign) { //removes leading 0, fixes sign (base)
32         for(int i = a.size() - 1; i > 0 && a[i] == '0'; --i) {
33             a.erase(a.begin() + i);
34         }
35         sign = (a.size() == 1 && a[0] == '0') ? 1 : newSign;
36         return (*this);
37     }
38     void operator=(std::string b) {
39         a = b[0] == '-' ? b.substr(1) : b;
40         reverse(a.begin(), a.end());
41         this->normalize(b[0] == '-' ? -1 : 1);
42     }
43     bool operator<(const Bigint &b) const {
44         if(sign != b.sign) {
45             return sign < b.sign;
46         }
47         if(a.size() != b.a.size()) {
48             return sign == 1 ? a.size() < b.a.size() : a.size() > b.a.size();
49         }
50         for(int i = a.size() - 1; i >= 0; --i) {
51             if(a[i] != b.a[i]) {
52                 return sign == 1 ? a[i] < b.a[i] : a[i] > b.a[i];
53             }
54         }
55         return false;
56     }
57     bool operator==(const Bigint &b) const {

```

```

58     return (a == b.a && sign == b.sign);
59 }
60 bool operator!=(const Bigint &b) const {
61     return !operator==(b);
62 }
63 Bigint operator+(Bigint b) {
64     if(sign != b.sign) {
65         return (*this) - (-b); //don't modify here
66     }
67     Bigint c;
68     for(int i = 0, carry = 0; i < a.size() || i < b.size() || carry; ++i) {
69         carry += (i < a.size() ? a[i] - 48 : 0) + (i < b.a.size() ? b.a[i] - 48
70 : 0);
71         c.a += (carry % 10 + 48);
72         carry /= 10;
73     }
74     return c.normalize(sign);
75 }
76 Bigint operator-() {
77     sign *= -1;
78     return (*this);
79 }
80 Bigint operator-(Bigint b) {
81     if(sign != b.sign) {
82         return (*this) + (-b);
83     }
84     int s = sign; sign = b.sign = 1;
85     if((*this) < b) {
86         return (b - (-(*this))).normalize(-s);
87     }
88     Bigint c;
89     for(int i = 0, borrow = 0; i < a.size(); ++i) {
90         borrow = (a[i] - borrow - (i < b.size() ? b.a[i] : 48));
91         c.a += (borrow >= 0 ? borrow + 48 : borrow + 58);
92         borrow = (borrow >= 0 ? 0 : 1);
93     }
94     return c.normalize(s);
95 }
96 Bigint operator*(Bigint b) {
97     Bigint c("0");
98     for(int i = 0, k = a[i] - 48; i < a.size(); ++i, k = a[i] - 48) {
99         while(k--) c = c + b;
100         b.a.insert(b.a.begin(), '0');
101     }
102     return c.normalize(sign * b.sign);
103 }
104 Bigint operator/(Bigint b) {
    assert(b != Bigint("0"));

```

```

105     if(b.size() == 1 && b.a[0] == '0') {
106         b.a[0] /= (b.a[0] - 48);
107     }
108     Bigint c("0"), d;
109     for(int j = 0; j < a.size(); ++j) {
110         d.a += "0";
111     }
112     int dSign = sign * b.sign; b.sign = 1;
113     for(int i = a.size() - 1; i >= 0; --i) {
114         c.a.insert(c.a.begin(), '0');
115         c = c + a.substr(i, 1);
116         while(!(c < b)) {
117             c = c - b, d.a[i]++;
118         }
119     }
120     return d.normalize(dSign);
121 }
122 Bigint operator%(Bigint b) {
123     assert(b != Bigint("0"));
124     if(b.size() == 1 && b.a[0] == '0') {
125         b.a[0] /= (b.a[0] - 48);
126     }
127     Bigint c("0");
128     b.sign = 1;
129     for(int i = a.size() - 1; i >= 0; --i) {
130         c.a.insert(c.a.begin(), '0');
131         c = c + a.substr(i, 1);
132         while(!(c < b)) c = c - b;
133     }
134     return c.normalize(sign);
135 }
136 friend std::istream& operator>>(std::istream &is, Bigint &integer) {
137     std::string input;
138     is >> input;
139     integer = input;
140     return is;
141 }
142 friend std::ostream& operator<<(std::ostream& os, const Bigint& integer) {
143     if (integer.sign == -1) {
144         os << "-";
145     }
146     for (int i = integer.a.size() - 1; i >= 0; --i) {
147         os << integer.a[i];
148     }
149     return os;
150 }
151 };
152

```

```

153 int main() {
154     Bigint a, b;
155     std::cin >> a >> b;
156     std::cout << a + b << '\n';
157     std::cout << a - b << '\n';
158     std::cout << a * b << '\n';
159     std::cout << a / b << '\n';
160     std::cout << a % b << '\n';
161     std::cout << (a == b ? "" : "not ") << "equal\n";
162     std::cout << "a is " << (a < b ? "" : "not") << "smaller than b\n";
163     std::cout << "the max number is:" << std::max(a, b) << '\n';
164     std::cout << "the min number is:" << std::min(a, b) << '\n';
165     return 0;
166 }

```

6.5 命令行

```

1 ***** bat *****
2 @echo off
3 g++ %1.cpp -std=c++20 -O2 -Wall -o %1 -D_GLIBCXX_DEBUG
4 ./%1 < in.txt > out.txt
5 @REM type out.txt
6 *****
7
8 ***** sh ***** chmod +x
9 #!/bin/bash
10 g++ -std=c++20 -O2 -Wall "$1.cpp" -o "$1" -D_GLIBCXX_DEBUG
11 ./"$1" < in.txt > out.txt
12 cat out.txt
13 *****
14
15 ***** sh ***** chmod +x
16 while true; do
17     ./gen > 1.in
18     ./std < 1.in > std.out
19     ./my < 1.in > my.out
20     if diff my.out std.out; then
21         echo ac
22     else
23         echo wa
24     break
25 fi
26 done
27 *****
28
29 ***** bat *****
30 @echo off
31 :loop

```

```

32 gen.exe > 1.in
33 std.exe < 1.in > std.out
34 my.exe < 1.in > my.out
35 fc my.out std.out > nul
36 if %errorlevel%==0 (
37     echo ac
38     goto loop
39 ) else (
40     echo wa
41     goto :eof
42 )
43 *****

```

7 附录

7.1 编译参数

-D_GLIBCXX_DEBUG : STL debugmode
 -fsanitize=address : 内存错误检查
 -fsanitize=undefined :UB 检查

7.2 OJ 测试

```

1 #include <bits/stdc++.h>
2 using i64 = long long;
3
4 int main() {
5     std::ios::sync_with_stdio(false);
6     std::cin.tie(nullptr);
7     volatile unsigned j = 1;
8     for(unsigned i = 1; i <= (unsigned)65E7; i++) {
9         j = 0;
10        j += i;
11        j *= i;
12        j /= i;
13    }
14    return 0;
15 }

```

Local	QOJ	AtCoder	LuoGu	Codeforces	Nowcoder
1000ms	1090ms	1120ms	1572ms	1718ms	1807ms

7.3 随机素数

979345007 986854057502126921
935359631 949054338673679153
931936021 989518940305146613
984974633 972090414870546877
984858209 956380060632801307

7.4 组合数学公式

性质 1:

$C_n^m = C_n^{n-m}$

性质 2:

$C_{n+m+1}^m = \sum_{i=0}^m C_{n+i}^i$

性质 3:

$C_n^m \cdot C_m^r = C_n^r \cdot C_{n-r}^{m-r}$

性质 4 (二项式定理):

$\sum_{i=0}^n (C_n^i \cdot x^i) = (1 + x)^n$

$\sum_{i=0}^n C_n^i = 2^n$

性质 5:

$\sum_{i=0}^n ((-1)^i \cdot C_n^i) = 0$

性质 6:

$C_n^0 + C_n^2 + \cdots = C_n^1 + C_n^3 + \cdots = 2^{n-1}$

性质 7:

$C_{n+m}^r = \sum_{i=0}^{\min(n,m,r)} (C_n^i \cdot C_m^{r-i})$

$C_{n+m}^n = C_{n+m}^m = \sum_{i=0}^{\min(n,m)} (C_n^i \cdot C_m^i), \quad (r = n \mid r = m)$

性质 8:

$m \cdot C_n^m = n \cdot C_{n-1}^{m-1}$

性质 9:

$\sum_{i=0}^n (C_n^i \cdot i^2) = n(n + 1) \cdot 2^{n-2}$

性质 10:

$\sum_{i=0}^n (C_n^i)^2 = C_{2n}^n$

7.5 常数表

<i>n</i>	log ₁₀ <i>n</i>	<i>n</i> !	<i>C</i> (<i>n</i> , <i>n</i> /2)	LCM(1... <i>n</i>)	<i>P_n</i>
2	0.30102999	2	2	2	2
3	0.47712125	6	3	6	3
4	0.60205999	24	6	12	5
5	0.69897000	120	10	60	7
6	0.77815125	720	20	60	11
7	0.84509804	5040	35	420	15
8	0.90308998	40320	70	840	22
9	0.95424251	362880	126	2520	30
10	1.00000000	3628800	252	2520	42
11	1.04139269	39916800	462	27720	56
12	1.07918125	479001600	924	27720	77
15	1.17609126	1.31e12	6435	360360	176
20	1.30103000	2.43e18	184756	232792560	627
25	1.39794001	1.55e25	5200300	26771144400	1958
30	1.47712125	2.65e32	155117520	1.444e14	5604
<i>P_n</i>	37338 ₄₀	204226 ₅₀	966467 ₆₀	190569292 ₁₀₀	1e9 ₁₁₄

max ω(*n*): 小于等于 *n* 中的数最大质因数个数
max *d*(*n*): 小于等于 *n* 中的数最大因数个数
π(*n*): 小于等于 *n* 中的数最大互质数个数

$n \leq$	10	100	1e3	1e4	1e5	1e6
$\max \omega(n)$	2	3	4	5	6	7
$\max d(n)$	4	12	32	64	128	240
$\pi(n)$	4	25	168	1229	9592	78498
$n \leq$	1e7	1e8	1e9	1e10	1e11	1e12
$\max \omega(n)$	8	8	9	10	10	11
$\max d(n)$	448	768	1344	2304	4032	6720
$\pi(n)$	664579	5761455	5.08e7	4.55e8	4.12e9	3.7e10
$n \leq$	1e13	1e14	1e15	1e16	1e17	1e18
$\max \omega(n)$	12	12	13	13	14	15
$\max d(n)$	10752	17280	26880	41472	64512	103680
$\pi(n)$	Prime number theorem: $\pi(x) \sim \frac{x}{\log(x)}$					